Warsaw University of Technology

FACULTY OF
POWER AND AERONAUTICAL ENGINEERING

Institute of Aeronautics and Applied Mechanics

Bachelor's diploma thesis

in the field of study Robotics and Automatic Control
and specialisation Robotics

Feedforward control of underactuated multibody systems in a co-simulation framework

Krzysztof Mazurek
student record book number 321055

thesis supervisor
dr hab. inż. Paweł Malczyk

WARSAW 2025

Feedforward control of underactuated multibody systems in a co-simulation framework

Abstract. This thesis investigates the feedforward control of planar underactuated multibody systems. The mechanisms were formulated using an absolute coordinate approach, considering exclusively revolute joints. The trajectory tracking control problem was addressed through the application of servo constraints, leading to an inverse dynamics formulation. The resulting set of differential-algebraic equations (DAE's) was derived and numerically solved using the backward Euler differentiation method, implemented in Python.

To enhance system stability, a simple feedback control loop was incorporated, and two distinct mechanisms were analyzed: a two-degree-of-freedom (2-DOF) rotating manipulator arm with a flexible coupling and a fully actuated four-link closed-loop 2-DOF mechanism. The study assessed the numerical accuracy of the computations, the robustness of trajectory tracking, and the influence of mechanical system parameters on dynamic behaviour. Furthermore, a co-simulation framework integrating Python, MATLAB, and MSC Adams was developed and described.

Keywords: Underactuated, Feedforward control, Multibody dynamics, Servo-constraint. Co-simulation

Sterowanie układami wieloczłonowymi typu underactuated z zastosowaniem sprzężenia kompensującego w przód w środowisku ko-symulacyjnym

Streszczenie. Niniejsza praca bada sterowanie wyprzedzające (feedforward) płaskich układów wieloczłonowych typu "underactuated". Mechanizmy zostały sformułowane przy użyciu podejścia opartego na współrzędnych ogólnych, uwzględniając wyłącznie przeguby obrotowe. Problem sterowania śledzeniem trajektorii został rozwiązany poprzez zastosowanie więzów programowych (servo- constraints), co doprowadziło do sformułowania zadania dynamiki odwrotnej. Otrzymany układ równań różniczkowo-algebraicznych (DAE) został wyprowadzony i numerycznie rozwiązany za pomocą metody różniczkowania wstecznego Eulera, zaimplementowanej w języku Python.

W celu poprawy stabilności układu wprowadzono prostą pętlę sterowania sprzężenia zwrotnego, a następnie przeanalizowano dwa różne mechanizmy: dwuprzegubowe ramię manipulatora (2-DOF) z elastycznym połączeniem oraz w pełni sterowany, zamknięty mechanizm czteroczłonowy o dwóch stopniach swobody (2-DOF). W badaniach oceniono dokładność obliczeń numerycznych, odporność sterowania śledzenia trajektorii oraz wpływ parametrów mechanicznych na zachowanie dynamiczne układu. Ponadto opracowano i opisano środowisko ko-symulacji integrujące język Python, MATLAB oraz program MSC Adams.

Słowa kluczowe: Underactuated, Sterowanie, Układy wieloczłonowe, Ko-symulacja

Oświadczam, że treść pracy dyplomowej w wersji drukowanej, treść pracy dyplomowej zawartej na nośniku elektronicznym (płycie kompaktowej) oraz treść pracy dyplomowej w module APD systemu USOS są identyczne.
I certify that the content of the printed version of the diploma thesis, the content of the electronic version of the diploma thesis (on a CD) and the content of the diploma thesis in the Archive of Diploma Theses (APD module) of the USOS system are identical.

.....
czytelny podpis studenta
legible signature of the student

Contents

1. Introduction

1.1. Underactuated systems overview

The precision and robustness of control in mechanical systems are among the most fundamental concerns in engineering disciplines. Consequently, it is generally required that all degrees of freedom of a mechanism be fully controlled. However, achieving full actuation often results in increased system weight and complexity while limiting the mechanical structure's full potential. In many cases, this traditional approach is either impractical or not optimal.

Interestingly, nature provides numerous examples of underactuated systems, virtually all animals are inherently underactuated mechanisms, as they possess more degrees of freedom than directly controlled actuators. They rely on internal dynamic properties and learned control strategies to achieve motion and stability. Somewhat inspired by these principles, the field of underactuated system control focuses on mechanisms with more degrees of freedom than available control inputs, aiming to enhance efficiency and performance while reducing actuation requirements.

Underactuation is particularly relevant in several engineering applications. In space industry, minimizing weight and complexity is crucial, making fully actuated designs impractical and excessively heavy. By the same token, all flying mechanisms fall in the category of underactuated mechanisms, since their all 6 degrees of freedom can never be controlled fully. Similarly, actuator failures in critical systems necessitate control strategies that allow continued operation with reduced actuation. In biomedical engineering, small-scale mechanisms, such as robotic prosthetics and surgical devices, must operate efficiently with minimal control inputs due to size constraints. And finally, strictly in the field of robotics the phenomenal achievements of humanoid robots, bipeds and nature inspired mechanisms are continuously improved.

On the highest level, field of underactuated mechanical systems is very complex. It forms a symbiosis of highly advanced mechanical designs and state of the art control algorithms. It is a popular and fast evolving topic of modern research and studies. The aim of this thesis is to take a step into this field. One of the fundamental topics will be studied, model based control algorithm. More precisely, feedforward control using servo constraints for trajectory tracking will be investigated.

1.2. Literature overview

In this section firstly a general overview will be taken on control algorithms used for underactuated mechanical systems. Then, more in depth look will be taken on the planar multibody systems, the focus of this thesis.

Following the idea proposed in [1] control strategies for underactuated systems can be split into two main categories: open-loop and closed-loop. First one is traditionally model

based. Solutions are often inexpensive and in perfect conditions can achieve fantastic trajectory tracking qualities. However, in real world they often can lack robustness. Since that, they often become a core for a more advanced closed loop solution that accounts for possible disturbances and inaccuracies of the model. The closed-loop systems are being discussed significantly more deeply. Again, [1] points out to methods such as partial feedback linearization, energy based approaches, sliding mode control, optimal control, feedback linearization approach as well as control methods based on fuzzy systems.
A set of lectures from MIT [2] shows a systematic development of complexity of control strategies. Significant part focuses on mechanisms in two dimensions, which are treated as an approachable way to display more advanced methodologies. Phenomena of biped robots are shown, such as modelling of dynamic contacts and walking principles. Advanced solutions using such as LQR regulator, system identification, robust control and imitation learning are displayed.

Topics of feedforward control in two dimensions acts as an approachable way to develop understanding of the methodology and is used extensively. Publications regarding modelling multibody systems this way are extensive, works of Parviz E. Nikravesh [3], [4] are the perfect example. Additionally, they propose systematic solutions that ease the software implementations of the discussed problems. The concept of using servo constraints has been developed deeply by Robert Seifried and Wojciech Blajer. Comprehensive book [5] and a PhD thesis [6] from the same research group explain the topic substantially, displaying the ideas precisely and exploring example simulation results. Additionally publications such as [7], [8], [9], [10] present solutions to a sample multibody systems described in a minimum set of coordinates. Important turns out to be publication [11], which describes approach using mixed set of coordinates.

Problems related to numerical differentiation, especially using Euler backward formula has been discussed deeply [12], [13]. Sources of information regarding software technicalities are explained precisely in official documentation such as Python `scipy` library [14].

1.3. Goal and primary assumption

The objective of this thesis was to examine the control properties of planar underactuated multibody systems through a feedforward control algorithm. The study considered mechanisms composed of perfectly rigid bodies connected by revolute joints only. The research followed a structured, multi-stage approach. Initially, a mathematical and computational framework was developed, encompassing the formulation and analysis of inverse dynamics problems for trajectory tracking by servo constraints in a minimal coordinate representation. Open-loop control solutions for fundamental multibody mechanisms were derived, and the corresponding differential-algebraic equations were numerically solved using Python software. The methodology was then extended to accommodate

mechanisms described in an absolute coordinate framework. Following that, a comprehensive computational toolkit, in a co-simulation form, for the feedforward control of underactuated multibody systems was designed, incorporating feedback control functionalities. Finally, validation experiments were conducted on selected underactuated mechanisms, and the results were systematically analyzed. Focus was drawn to the computational error discussion, feedback loop influence and relation between control software setup and mechanical properties of the multibody system.

2. Modelling of planar multibody systems

2.1. Definitions and assumptions

To study kinematics of multibody systems reference frame definitions are needed. The global coordinate system is set as x-y. The local, fixed to the body reference frame is denoted by ξ- η. In this thesis it will be always located in the center of mass of each body.

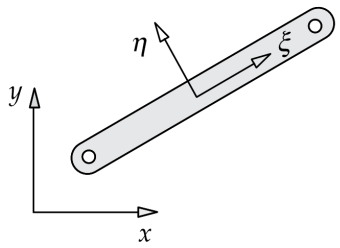


Figure 2.1. Coordinates definition [4]

The following assumptions and naming convention will be chosen for the following paragraphs:
m_i - number of control inputs
m_j- number of degrees of freedom constrained by mechanical joints
n - degrees of freedom
k - number of bodies

Only planar mechanisms with revolute joints will be discussed. In such mechanisms, all revolute joint axes are parallel to each other and perpendicular to the plane of motion. They can be classified as holonomic constraints. It is assumed that bodies are rigid, distances between particles in the body stay unchanged.

2.2. Governing equation of motion

Based on Newton-Euler formalism mechanics the multibody equations of motion can be described by the following equation,

$$\mathbf{M}\ddot{\mathbf{q}} = \mathbf{f} \tag{1}$$

here, **M** is a general mass matrix, **q** a coordinate vector, **f** a combined force vector due to applied, centrifugal, Coriolis and gyroscopic effects. The dimensions of the equations depend on the coordinate system chosen, in the next paragraph briefly a minimal coordinate formulation will be discussed, in the following one with more detail the Cartesian coordinate formulation will be investigated.

2.3. Minimum set coordinates

The most principle and basic way of formulating equations of motion of mechanical multibody systems is the use of Euler-Lagrange equations. Based on the energy conservation law where T is potential energy and V is kinetic energy.

$$L = T - V \tag{2}$$

In the basic form, the Euler Lagrange Equation is described below.

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\mathbf{q}}_i}\right) - \frac{\partial L}{\partial \mathbf{q}_i} = 0 \tag{3}$$

However, considering energy dissipation due to elements such as dampers an additional term is introduced. Considering the Raleigh dissipation function the updated equation looks as follows:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\mathbf{q}}_i}\right) - \frac{\partial L}{\partial \mathbf{q}_i} = -\frac{\partial R}{\partial \dot{\mathbf{q}}_i}, \tag{4}$$

where R is defined:

$$R(\dot{\mathbf{q}}) \equiv \frac{1}{2}\sum_{i=1}^{n} b_i \dot{\mathbf{q}}_i^2. \tag{5}$$

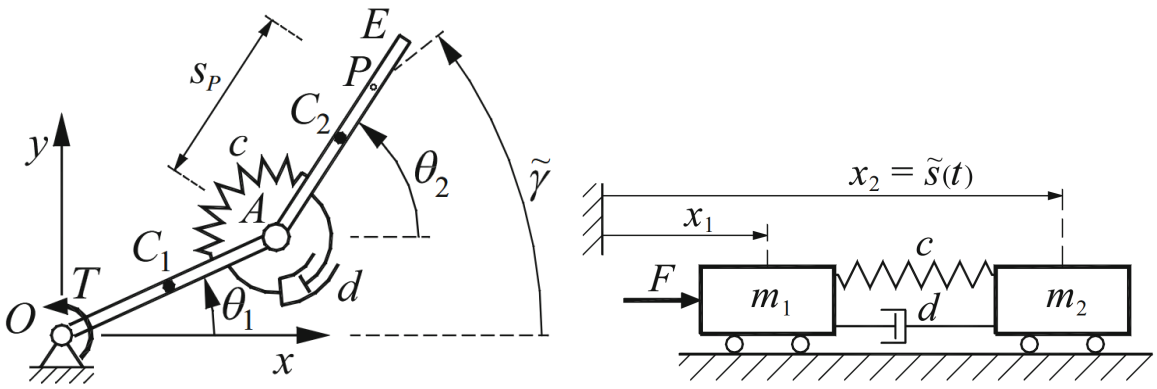
and b_i are damping coefficients related to forces acting on i-th body.

In general the resulting equation can be transformed to the form shown in equation (1). The dimension of the equation are equal to the number of degrees of freedom- k. So, the shapes of the matrixes is as follows: $\mathbf{M}_{k\times k}$, $\mathbf{q}_{k\times 1}$, $\mathbf{f}_{k\times 1}$. The equations of motion are made in the minimum set of coordinates. In 2.2 shown are proposed coordinates choices for the systems with two degrees of freedom, θ_1, θ_2 for the figure (a); x_1, x_2 for the figure (b). The relatively high understanding of the system is needed, followed by manual differentiation of equations of conservation of energy. Each unique mechanism has its own properties that need to be taken into consideration. Developing equations of motion manually turns out to be highly time-consuming and complicated. However, when discussing basic planar multibody systems this approach is sufficient and results in a compact solution that is easy to analyse. Therefore many publications use the set of minimum coordinates to discuss the topics of feedforward control of multibody systems [7], [15], [8], [2]. The equation (1) contains all the information about the mechanical properties of the system, kinematic constraints included. As an example, a solution to the system shown in Fig. 2.2(a) is proposed [9]:

$$\mathbf{M} = \begin{bmatrix} J_{C1} + m_1 s_1^2 + m_2 l_1^2 & m_2 s_2 l_1 \cos(\theta_1 - \theta_2) \\ m_2 s_2 l_1 \cos(\theta_1 - \theta_2) & J_{C2} + m_2 s_2^2 \end{bmatrix},$$

$$\mathbf{f} = \begin{bmatrix} -m_2 s_2 l_1 \dot{\theta}_2^2 \sin(\theta_2 - \theta_1) \\ m_2 s_2 l_1 \dot{\theta}_1^2 \sin(\theta_2 - \theta_1) \end{bmatrix} - \begin{bmatrix} c(\theta_2 - \theta_1) + d(\dot{\theta}_2 - \dot{\theta}_1) \\ -c(\theta_2 - \theta_1) - d(\dot{\theta}_2 - \dot{\theta}_1) \end{bmatrix},$$

where J are moments of inertia, c, d are spring stiffness and damping coefficients respectively. As mentioned above, the system is described in minimal number of coordinates.



(a) Two-link manipulator [8] **(b)** Two-mass cart system [8]

Figure 2.2. Example choices of dependent coordinates for underactuated systems

Due to a relatively high complexity of the equations it is hard to generalize them to describe any type of a planar multibody mechanism. Therefore a different approach, using Cartesian coordinates is needed.

2.4. Formulation in absolute coordinates

The use of absolute coordinates approach varies significantly from the one proposed in the previous paragraph. Each body has its own local coordinate system ξ- η, for convenience located in the centre of mass of each body, simplifying therefore calculations of moments of inertia. From now on, the absolute coordinate system will be referred to as simply $x - y$. To sum up, the position for each centre of mass can be described with two coordinates x_i, y_i, also referred to as $\tilde{\mathbf{q}}_i$. To specify the orientation of the body fully, another variable is needed, the angle of rotation of the local coordinate system in global frame of reference- α. Following that, the position and orientation of each body can be described by :

$$\mathbf{q}_i = \begin{bmatrix} x_i & y_i & \alpha_i \end{bmatrix}^T, \tag{6}$$

$$\tilde{\mathbf{q}}_i = \begin{bmatrix} x_i & y_i \end{bmatrix}^T. \tag{7}$$

Considering a multibody system composed of i parts, systematic approach to describe position and velocities of all bodies accordingly can be formulated:

$$\mathbf{q}_{3k\times 1} = \begin{bmatrix} \mathbf{q}_1^T & \mathbf{q}_2^T & \dots & \mathbf{q}_k^T \end{bmatrix}^T \tag{8}$$

$$\mathbf{v}_{3k\times 1} = \begin{bmatrix} \mathbf{v}_1^T & \mathbf{v}_2^T & \dots & \mathbf{v}_k^T \end{bmatrix}^T \tag{9}$$

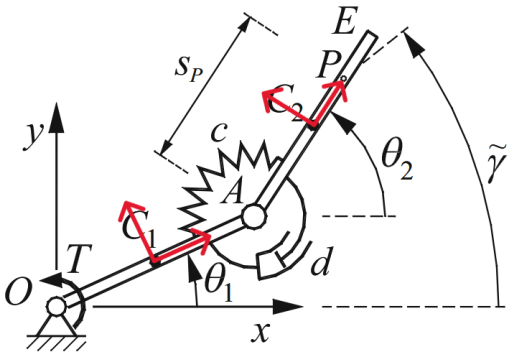


Figure 2.3. CM local coordinate system in red, angle of rotation described by θ

Using the absolute coordinates approach a new variant of equation of motion can be derived 10, where $\mathbf{C}$ is a Jacobian matrix $\frac{\partial \mathbf{\Phi}}{\partial \mathbf{q}}$ of material constrain equations and $\boldsymbol{\lambda}$ is a vector of Lagrange multipliers indicating the magnitude of constraint loads at joints. The $\mathbf{f}$ vector now is responsible for only external forces acting on bodies, such as gravity or spring-damper connections. The result is a set of $3k$ equations of motion combined with m_j equation of constraints.

$$\begin{aligned} \mathbf{M}\ddot{\mathbf{q}} - (\mathbf{f} + \mathbf{C}^T\boldsymbol{\lambda}) &= \mathbf{0}, \\ \mathbf{\Phi} &= \mathbf{0} \end{aligned} \tag{10}$$

In this approach the mass matrix can be derived systematically. It is a diagonal matrix where m_i and I_i are the mass and a centre moment of inertia of i-th body.

$$\mathbf{M}_i = \begin{bmatrix} m_i & 0 & 0 \\ 0 & m_i & 0 \\ 0 & 0 & I_i \end{bmatrix} \tag{11}$$

For the system of k bodies the mass matrix is defined :

$$\mathbf{M}_{3k\times 3k} = \begin{bmatrix} \mathbf{M}_1 & 0 & 0 & 0 \\ 0 & \mathbf{M}_2 & \dots & \dots \\ \vdots & \vdots & \ddots & \dots \\ 0 & 0 & 0 & \mathbf{M}_k \end{bmatrix} \tag{12}$$

It is important to add, that the order of bodies is unrelated to their connection order, it must only be used systematically throughout the solution. Next, the $\mathbf{f}_i$ vector represents the 2 external forces acting in x, y directions and a torque acting on each body is derived. It can be generalized to a complete multibody system.

$$\mathbf{f}_{3k\times 1} = \begin{bmatrix} \mathbf{f}_1^T & \mathbf{f}_2^T & \dots & \mathbf{f}_k^T \end{bmatrix}^T \tag{13}$$

The **C** matrix is a Jacobian matrix of mechanical constraint equations. In the thesis, only holonomic rotational joints are considered. They constrain the body in the x and y direction therefore they can be classified as second order ${}^{(r,2)}\mathbf{\Phi}$.

$$\mathbf{\Phi} = \mathbf{r}_i^P - \mathbf{r}_j^P = \mathbf{0} \tag{14}$$

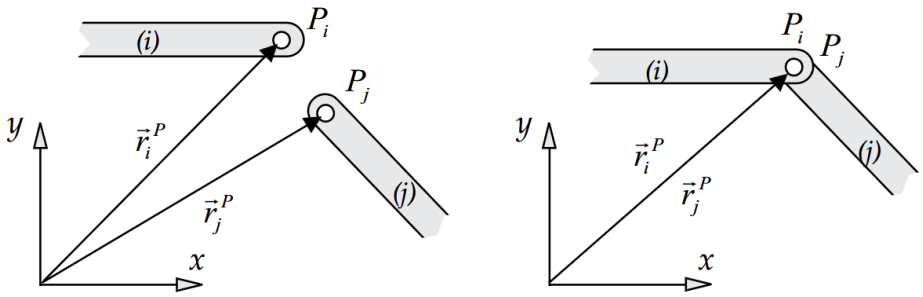


Figure 2.4. Revolute joint [4]

Given the rotation matrix $\mathbf{R}(\alpha)$ and assuming that vector $\mathbf{s}_P^i$ is in coordinate system of i-th body and points from the center of mass to a point of connection, constraint matrix $\mathbf{\Phi}$ [16] can be derived.

$$\mathbf{R}(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix} \tag{15}$$

$$\mathbf{\Phi}_{ij} = \tilde{\mathbf{q}}_i + \mathbf{R}(\alpha_i)\mathbf{s}_P^i - \tilde{\mathbf{q}}_j - \mathbf{R}(\alpha_j)\mathbf{s}_P^j \tag{16}$$

A general constrain matrix convention needed to be derived to express mechanical constraints in an ordered way. The data structure used in Adams software was taken into consideration. Let's assume that a multibody system has $l = m_j/2$ rotational joint connections. Each joint is composed of two bodies B_i and B_j. Let's assume that a body B_i contains a set number of points

$$P_i = \{P_{i1}, P_{i2}, P_{i3} \dots P_{i*}\}$$

In other words $P_i \in B_i$. In this way a joint can be described by:

$$J_{i*j*}(\{B_i, P_{i*}\}, \{B_j, P_{j*}\}).$$

Therefore:

$$\mathbf{\Phi}(J_{i*j*}) = \tilde{\mathbf{q}}_i + \mathbf{R}(\alpha_i)\mathbf{s}^i_{Pi*} - \tilde{\mathbf{q}}_j - \mathbf{R}(\alpha_j)\mathbf{s}^j_{Pj*} \tag{17}$$

To increase readability "$*$" symbolizes appropriate connection point on a body. The general $\mathbf{\Phi}$ matrix is defined for all existing joints in the system in an arbitrary order.

$$\mathbf{\Phi}_{2l\times 1} = \begin{bmatrix} \mathbf{\Phi}_1 & \mathbf{\Phi}_2 & \dots & \mathbf{\Phi}_l \end{bmatrix}^T \tag{18}$$

It is important to point out the properties the connections of elements with ground. The rotation of the coordinate system would lack logical coherence in this sense. Assuming that ground has index i, $\tilde{\mathbf{q}}_i + \mathbf{R}(\alpha_i)\mathbf{s}^i_{Pi*} \rightarrow \mathbf{s}^0_{P0*}$.

Further, a Jacobian matrix $\mathbf{\Phi_q} = \mathbf{C}$ of the constraint equations is made. After differentation of the constraint equations with respect to consecutive coordinates the following equation was achieved

$$\mathbf{\Phi}_{\mathbf{q}(ij)} = \begin{bmatrix} \mathbf{I}_{2\times 2} & \mathbf{\Omega R}(\alpha_i)\mathbf{s}^i_{Pi} & -\mathbf{I}_{2\times 2} & -\mathbf{\Omega R}(\alpha_j)\mathbf{s}^j_{Pj} \end{bmatrix} = \begin{bmatrix} \tilde{\mathbf{\Phi}}_{\mathbf{q}(i)} & -\tilde{\mathbf{\Phi}}_{\mathbf{q}(j)} \end{bmatrix} \tag{19}$$

where,

$$\mathbf{\Omega} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}. \tag{20}$$

A proposed solution to a general constraint Jacobian matrix is described below. The order of joint connection is taken into consideration, the element $\tilde{\mathbf{\Phi}}_{\mathbf{q}(i)}$ corresponding to the first body is taken with the positive sign, the latter with negative. It allows for a systematic and robust solution. Again, it is important to specify the properties of the ground, a part that is fixed in relation to the global coordinates and assigned the zero coordinates. The equation (21) shows an example approach to a system with multiple connections, they are left in an unordered way to express the robustness of the approach.

$$\mathbf{\Phi}_{\mathbf{q}(2l\times 3k)} = \begin{array}{c} \\ \mathbf{\Phi}_{0,2} \\ \mathbf{\Phi}_{1,2} \\ \mathbf{\Phi}_{3,2} \\ \\ \mathbf{\Phi}_{l,0} \end{array} \begin{array}{c} \overbrace{\phantom{\mathbf{0}_{2\times 3}}}^{\mathbf{q}_1} \\ \mathbf{0}_{2\times 3} \\ \tilde{\mathbf{\Phi}}_{\mathbf{q}(1)} \\ \mathbf{0}_{2\times 3} \\ \vdots \\ \mathbf{0}_{2\times 3} \end{array} \begin{array}{c} \overbrace{\phantom{\mathbf{0}_{2\times 3}}}^{\mathbf{q}_2} \\ \mathbf{0}_{2\times 3} \\ -\tilde{\mathbf{\Phi}}_{\mathbf{q}(2)} \\ -\tilde{\mathbf{\Phi}}_{\mathbf{q}(2)} \\ \vdots \\ \mathbf{0}_{2\times 3} \end{array} \begin{array}{c} \overbrace{\phantom{\mathbf{0}_{2\times 3}}}^{\mathbf{q}_3} \\ -\tilde{\mathbf{\Phi}}_{\mathbf{q}(2)} \\ \mathbf{0}_{2\times 3} \\ \tilde{\mathbf{\Phi}}_{\mathbf{q}(3)} \\ \vdots \\ \mathbf{0}_{2\times 3} \end{array} \begin{array}{c} \\ \dots \\ \dots \\ \dots \\ \ddots \\ \dots \end{array} \begin{array}{c} \overbrace{\phantom{\mathbf{0}_{2\times 3}}}^{\mathbf{q}_l} \\ \mathbf{0}_{2\times 3} \\ \mathbf{0}_{2\times 3} \\ \mathbf{0}_{2\times 3} \\ \vdots \\ \tilde{\mathbf{\Phi}}_{\mathbf{q}(l)} \end{array} \tag{21}$$

3. Feedforward control using servo constraints

A multibody system should track a specified trajectory based on an inverse dynamic model in a form of feedforward algorithm, which provides necessary control inputs. As opposed to widely discussed fully actuated systems, the control of underactuated systems is significantly more complicated. A proposed in [7], [11], [5] solution to the trajectory tracking problem is the use of servo constraints, also called program constraints. The open-loop control can be then supported by a feedback loop providing a more robust algorithm. Considering the addition of control inputs an updated equation of motion is derived, where $\mathbf{B}^T_{m_i \times 3k}$ matrix projects the control input forces and torques $\mathbf{u}_{m_i \times 1}$ to appropriate coordinates.

$$\begin{aligned} \mathbf{M}\ddot{\mathbf{q}} - (\mathbf{f} + \mathbf{B}^T\mathbf{u} + \mathbf{C}^T\boldsymbol{\lambda}) &= \mathbf{0}, \\ \boldsymbol{\Phi} &= \mathbf{0} \end{aligned} \tag{22}$$

3.1. Servo constraints

The prescribed in time outputs $\tilde{y}(t)$, are treated as performance goals of the system at a specified point P. *Output trajectory tracking deals with the problem of forcing the physical system to reproduce or follow a predefined desired reference output trajectory* $\tilde{y}(t)$[5].
If a system is fully actuated or overactuated f=n outputs need to be specified. However, when dealing with underactuated systems number of outputs should be equal to the number of inputs, leading to f=m <n specified outputs called servo or program constraints. As a result a servo constraint are defined

$$\boldsymbol{\Psi}(\mathbf{q}, t) = \mathbf{h}(\mathbf{q}) - \tilde{\mathbf{y}}(t) = \mathbf{0}, \tag{23}$$

where $\mathbf{h}(\mathbf{q})$ defines a position of a tracking point and a $\tilde{\mathbf{y}}(t)$ is a prescribed in time trajectory. It should have a sufficiently smooth acceleration curve, a starting position at time t_0 and ending at time t_f.

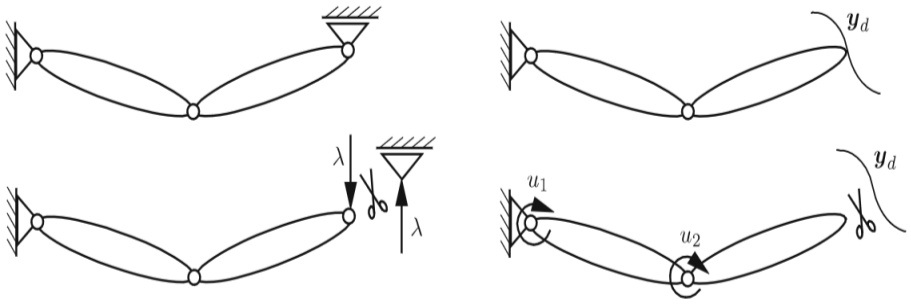


Figure 3.1. Comparison between servo and mechanical constraint [5]

Taking the equation(23) into consideration and applying first order system reduction the following set of equations is formed.

$$\begin{aligned} \dot{\mathbf{q}} &= \mathbf{v} \\ \mathbf{M}\dot{\mathbf{v}} &= \mathbf{f} + \mathbf{B}^T\mathbf{u} + \mathbf{C}^T\boldsymbol{\lambda} \\ \boldsymbol{\Phi} &= \mathbf{0} \\ \boldsymbol{\Psi} &= \mathbf{0} \end{aligned} \tag{24}$$

With $6k$ unknown positions and velocities $(\mathbf{q}, \dot{\mathbf{q}})$, m_j unknown Lagrange multipliers $(\boldsymbol{\lambda})$ and m_i control inputs $(\mathbf{u})$ the system is a high order differential algebraic equation (DEA).

In a logical sense program constraints look very similar to mechanical constrain, where the $\mathbf{C}^T\boldsymbol{\lambda}$ correspond to $\mathbf{B}^T\mathbf{u}$. However, the idea behind them is significantly more complicated. Mechanical constraints in the classical sense (hard surfaces, slip-less rolling contacts, etc.) have a reaction force acting perpendicular to the constraint manifold, therefore are classified as orthogonal. The servo constraints however, can act in arbitrary orientation to the control inputs, resulting in orthogonal, mixed or even tangential orientation. For systems with multiple inputs and outputs a mixed orthogonal- tangential configuration may exist, where only some outputs can be influenced directly while other only can be indirectly. A measure of control singularity can be measured by defficiency in rank p of equation [8]

$$\mathbf{P} = \mathbf{S}\mathbf{M}^{-1}\mathbf{B}, \tag{25}$$

where $\mathbf{S}$ is a Jacobian matrix of servo constraints defined as follows:

$$\mathbf{S}_{m\times n} \equiv \frac{\partial \boldsymbol{\Psi}}{\partial \mathbf{q}} = \frac{\partial \mathbf{h}}{\partial \mathbf{q}}. \tag{26}$$

There are three possible servo-constraint types:

1. $p = m$- orthogonal
2. $0 < p < m$- orthogonal- tangential
3. $p = 0$- tangential

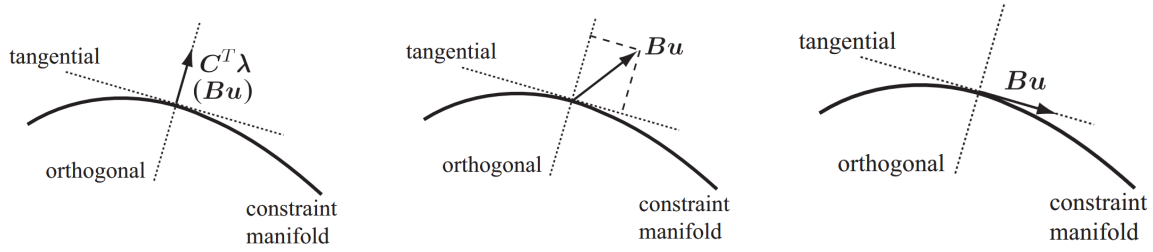


Figure 3.2. Configurations of servo constraints

As a conclusion to that it is clear that not all chosen trajectories $\tilde{\mathbf{y}}(t)$ of arbitrary points are possible to be succesfully followed.

3.2. Projection approach

Inverse model DAEs can be of high index, which makes the numerical solution ill-conditioned. Therefore a index reduction methods are very beneficiary. Proposed in [7] is a differentiation index reduction based on projection of system into complementary subspaces. Following this method system of equations (24) can be projected into three subspaces: mechanical constraint **C**, servo constraint **S** and orthogonal to both of them **D**. Firstly servo constrain needs to be differentiated twice in respect to time.

$$\dot{\boldsymbol{\Psi}}(\mathbf{q}, t) = \mathbf{S}(\mathbf{q})\dot{\mathbf{q}} - \dot{\tilde{\mathbf{y}}}(t) = \mathbf{0} \tag{27}$$

$$\ddot{\boldsymbol{\Psi}}(\mathbf{q}, t) = \mathbf{S}(\mathbf{q})\ddot{\mathbf{q}} + \boldsymbol{\xi}(\dot{\mathbf{q}}, \mathbf{q}, t) = \mathbf{0} \tag{28}$$

Where $\boldsymbol{\xi}$ is defined below.

$$\boldsymbol{\xi} = \dot{\mathbf{S}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \ddot{\tilde{\mathbf{y}}}(t) \tag{29}$$

Following that, the same steps are taken to derive the mechanical constraint derivatives.

$$\dot{\boldsymbol{\Phi}}(\mathbf{q}, t) = \mathbf{C}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0} \tag{30}$$

$$\ddot{\boldsymbol{\Phi}}(\mathbf{q}, t) = \mathbf{C}(\mathbf{q})\ddot{\mathbf{q}} + \boldsymbol{\delta}(\dot{\mathbf{q}}, \mathbf{q}, t) = \mathbf{0} \tag{31}$$

And $\boldsymbol{\delta}$ is defined as below.

$$\boldsymbol{\delta} = \dot{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} \tag{32}$$

The complementary subspace has subsequent properties.

$$\begin{bmatrix} \mathbf{S} \\ \mathbf{C} \end{bmatrix} \mathbf{D} = \mathbf{0} \iff \begin{cases} \mathbf{SD} = \mathbf{0} \\ \mathbf{CD} = \mathbf{0} \end{cases} \implies \begin{cases} \mathbf{D}^T\mathbf{S}^T = \mathbf{0} \\ \mathbf{D}^T\mathbf{S}^T = \mathbf{0} \end{cases}$$

Following that, equation of motion can be split into three complementary subspaces.

$$\begin{bmatrix} \mathbf{D}^T \\ \mathbf{SM^{-1}} \\ \mathbf{CM{-}1} \end{bmatrix} (\mathbf{M}\ddot{\mathbf{q}} - \mathbf{f} - \mathbf{B}^T - \mathbf{C}^T\boldsymbol{\lambda}) = \mathbf{0} \tag{33}$$

Further, equations (29) and (32) are inserted, as a result the reduction of the index occurs. The relation $\mathbf{D}^T\mathbf{S}^T = 0$ is also taken into consideration. Finally, a set of reduced-

rank DAE equation is derived.

$$\begin{aligned}
\dot{\mathbf{q}} - \mathbf{v} &= \mathbf{0} \\
\mathbf{D}^T \mathbf{M}\dot{\mathbf{v}} - \mathbf{D}^T(\mathbf{f} + \mathbf{B}^T) &= \mathbf{0} \\
\boldsymbol{\xi} - \mathbf{S}\mathbf{M}^{-1}(\mathbf{f} + \mathbf{B}^T + \mathbf{C}^\mathbf{T}\boldsymbol{\lambda}) &= \mathbf{0} \\
\boldsymbol{\delta} - \mathbf{C}\mathbf{M}^{-1}(\mathbf{f} + \mathbf{B}^T + \mathbf{C}^\mathbf{T}\boldsymbol{\lambda}) &= \mathbf{0} \\
\boldsymbol{\Phi} &= \mathbf{0} \\
\boldsymbol{\Psi} &= \mathbf{0}
\end{aligned} \tag{34}$$

3.2.1. Orthogonal matrix construction

Three main approaches were considered to derive the **D** matrix. Firstly a arithmetic transformations or almost- guessing can produce a correct result for uncomplicated systems, especially used in the systems described in minimum coordinates. However, a more solid and systematic solution was described in [7]. Firstly combined mechanical and servo Jacobian matrixes are factorized

$$\begin{bmatrix} \mathbf{S} \\ \mathbf{C} \end{bmatrix} = \Big[\mathbf{U} : \mathbf{W} \Big],$$

so that **U** is $(m_j + m_i) \times (3k - m_j - m_i)$ and **W** is $(m_j + m_i) \times (m_j + m_i)$. It is assumed that $det(\mathbf{W}) \neq 0$. It is important to add that for a fully actuated system ($m_i + m_j = 3k$) the **U** matrix is empty. Next **D** can be deduced.

$$\mathbf{D} = \Big[\mathbf{I} : (-\mathbf{W}^{-1}\mathbf{U})^T \Big]^T$$

A third, alternative approach, uses the properties of singular value decomposition (SVD) [16] where

$$\begin{bmatrix} \mathbf{S} \\ \mathbf{C} \end{bmatrix} = \mathbf{M}\Sigma\mathbf{V}^*.$$

Assuming the rank of the stacked matrixes is k, then the null space is spanned by the column vectors of $\mathbf{V}^*$, starting from k-th column. This approach is easily implemented in python code with the help of `numpy.linalg.svd` method.

3.3. Euler backward integration

There are a variety of possible numerical methods to solve DAEs. The approach taken in this thesis is the use of Euler backward differentation method to solve equations (34). In this way position and velocity derivatives can approximated be by their backward differences. At a time $t_{n+1} = t_n + \Delta t$:

$$\begin{aligned}\dot{\mathbf{q}} &= \frac{\mathbf{q}_{n+1}-\mathbf{q}_n}{\Delta t}\\ \dot{\mathbf{v}} &= \frac{\mathbf{v}_{n+1}-\mathbf{v}_n}{\Delta t}.\end{aligned} \tag{35}$$

The values $\mathbf{q}_{n+1}, \mathbf{v}_{n+1}, \mathbf{u}_{n+1}, \boldsymbol{\lambda}_{n+1}$ are solved following the solution of the discretized equations.

$$\begin{aligned}\frac{\mathbf{q}_{n+1}-\mathbf{q}_n}{\Delta t} - \mathbf{v}_{n+1} &= \mathbf{0}\\ \mathbf{A_1}(\mathbf{q}_{n+1})\frac{\mathbf{v}_{n+1}-\mathbf{v}_n}{\Delta t} - \mathbf{a_2}(\mathbf{v}_{n+1}, \mathbf{q}_{n+1}, \mathbf{u}_{n+1}, t_{n+1}) &= \mathbf{0}\\ \mathbf{b_1}(\mathbf{v}_{n+1}, \mathbf{q}_{n+1}, t_{n+1}) + \mathbf{b_2}(\mathbf{v}_{n+1}, \mathbf{q}_{n+1}, \mathbf{u}_{n+1}, \boldsymbol{\lambda}_{n+1}, t_{n+1}) &= \mathbf{0}\\ \mathbf{c_1}(\mathbf{v}_{n+1}, \mathbf{q}_{n+1}) + \mathbf{c_2}(\mathbf{v}_{n+1}, \mathbf{q}_{n+1}, \mathbf{u}_{n+1}, \boldsymbol{\lambda}_{n+1}, t_{n+1}) &= \mathbf{0}\\ \mathbf{d_1}(\mathbf{q}_{n+1}) &= \mathbf{0}\\ \mathbf{e_1}(\mathbf{q}_{n+1}, t_{n+1}) &= \mathbf{0}\end{aligned} \tag{36}$$

Euler backward method is an implicit method, it has therefore a wide region of stability. For the small enough time step h a local truncation error (LTE) of the method is approximately proportional to a constant vector multiplied by $\mathcal{O}(h^2)$ [13]. The RMS error is $\mathcal{O}(h)$, meaning that generally accumulative error increases linearly with the step size. Since that, a chosen step size can influence the result greatly. Euler forward method seems to have a dampening effect on sinusoidal shaped functions [17], this important to note, since many mechanical systems can share similar properties.

3.4. Feedforward- feedback loop

The excellence of the model- based control algorithm proposed is this chapter is the accuracy of the outputs, leading to very precise trajectory tracking properties. However, in a real world situation there are always small disturbances that, if left unaccounted for, would lead often to significant errors. Computation error is another potential cause for the inaccuracy of the results. Since that, a two-design degrees of freedom control structure, composed of a feedforward control and a feedback control is proposed shown in 3.3. The control input generation is based on the model inversion algorithm while the feedback controller accounts for potential disturbances and errors. In this way those two systems can run independently to each other. Main controller works either on-line or off-line. Feedback is based on the difference between a chosen state variables calculated by the generator and a real values measured in a multibody system, for example with the use of sensors. It can be based on a PID scheme or simpler solutions such as PD or even P

controllers. The combined control input is described by:

$$\mathbf{u} = \mathbf{u}_d + \mathbf{u}_c \tag{37}$$

The advantage of this solution is its simplicity and ability to decouple the two control elements. In the next chapter proposed is a co-simulation environment that demonstrates the two-block virtual control and testing environment.

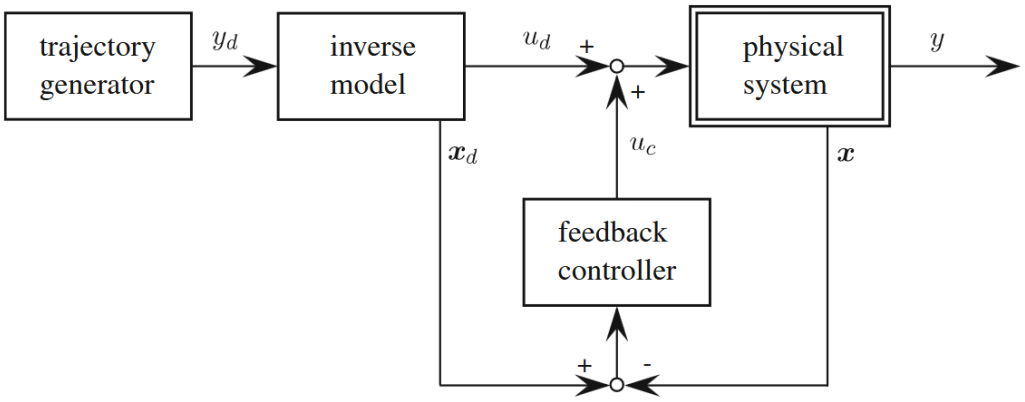


Figure 3.3. Two degrees of freedom control structure

Servo constraints are used in feedforward control to track a predefined trajectory in underactuated multibody systems. A feedforward inverse dynamic model provides necessary control inputs, while a feedback loop enhances robustness. Servo constraints, unlike mechanical constraints, can act in various orientations, affecting control effectiveness. To improve numerical stability, a projection-based index reduction method is applied. Finally, a feedforward-feedback control structure is proposed to mitigate disturbances and computational errors, ensuring precise trajectory tracking.

4. Co-simulation environment

Making a feedforward multibody control toolkit was one of the aims of this thesis. A co- simulation structure was chosen in a way to provide a convenient workflow through the design process, high reliability and result validity. The setup was made in a way to closely emulate work in real world physical conditions.

As an objective, the main software was to be developed in Python. Following this assumption, two possible configurations were considered. Firstly a fully python environment. Real world conditions were to be simulated using PyDy [18], PyBullet[19] or pydrake[20] libraries. A significant advantage of it was simplicity of communication and architecture, as well as modernity. Some design elements, such as kinematic joints and forces would have to be designed manually leading to potential mistakes. Second option, the chosen one, was to extend the possibilities of Msc Adams software by adding a feedforward control toolkit to it.

Environment is composed of three essential software with additional CAD software of any kind. Msc Adams is used as a virtual testing environment. It allows the user to connect parts with joints in a simple way and add various forces to the system. All mechanism states can be observed with the use of sensors. It provides an accurate multibody forward dynamics solver and a friendly user interface. Built in it has a control toolkit enabling setting up a co- simulation with Matlab/ Simulink environment. In a relatively straightforward way, Simmulink blocks can be connected with python scripts which, are used to calculate the control torque values.

Firstly a CAD model shall be made. Obviously in a three-dimensional form, however, taking into account that the toolkit is designed for controlling multibody systems with revolute joints in two dimensions . Only the dimensional properties will be exported, so the material properties will need to be updated in Adams. At the end of the design process a assembly should be exported preferably in `parasolid` format. After importing the model to Adams appropriate revolute connections should be made between corresponding parts. Furthermore, external forces and spring connections should be created. In the next step, state variables ought to be chosen corresponding to control inputs and outputs of the model. Inputs are typically torques. Outputs can be treated as sensors, either for the control point or other positions or velocities. It is essential to contain time as on of the control variables as it synchronizes the whole simulation environment. Next, the Control Plant can be exported, leading to the creation of Simulik- Adams co- simulation environment, supported by supplementary files.

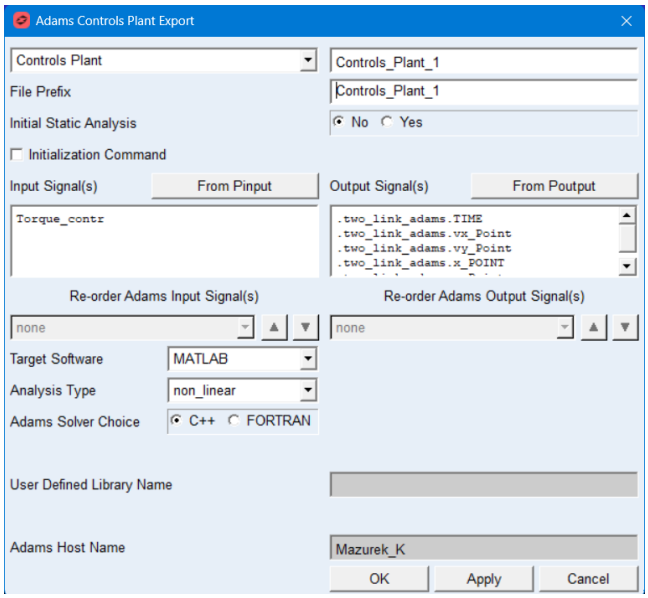


Figure 4.1. Adams plant export example setup

At this point the connection of three software component is needed. The basic structure is shown in Figure 4.2. The environment can be split into two main parts:

- Main control block: python open- loop feedforward trajectory tracking controller coupled with Simulink proportional feedback controller. Copy of a `.adm` file containing all information about physical properties of the model.
- Virtual environment: Adams model based on the CAD file.

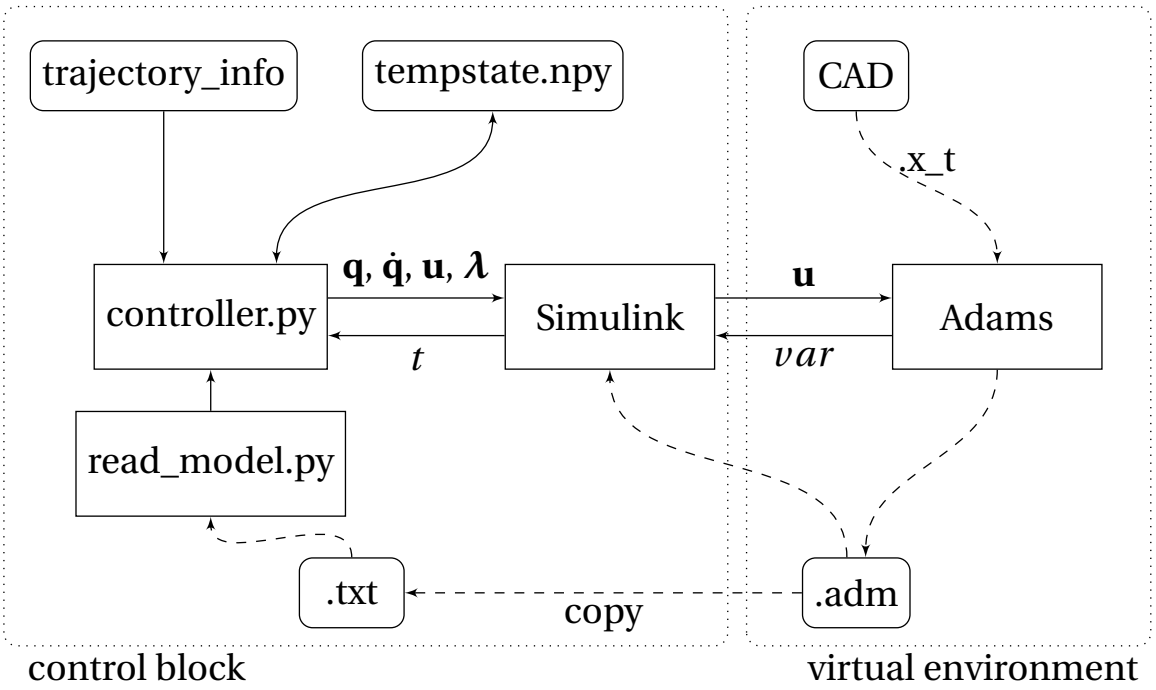


Figure 4.2. Co-simulation structure

The solution enables to independently control all properties of the system. Before the start of the simulation .adm file is copied and read by python `read_model.py` script. It creates a following simplified class structure that is corresponding to the one provided by Adams,and contains all physical properties of the model:

- `Part_Collection []`
  - `Part`
    - `Mass`
    - `Inertia`
    - `CM coordinates`
    - `Marker []`
    - `s_vector []`
- `Revolute_joint []`
  - `Part_a`
  - `Part_b`
  - `s_a`
  - `s_b`

Then, in a automatic way following vectors and matrixes are constructed: $\mathbf{q}$, $\dot{\mathbf{q}}$, $\mathbf{f}$, $\boldsymbol{\Phi}$, $\boldsymbol{\Phi}_\mathbf{q}$, $\dot{\boldsymbol{\Phi}}_\mathbf{q}$, $\mathbf{M}$. They are independent form real model, the .txt file can be edited to simulate discrepancy between the provided model of the mechanism and its real physical properties. Next, servo constraint and desired trajectories need to be specified for a particular mechanism: $\boldsymbol{\Psi}$, $\boldsymbol{\Psi}_\mathbf{q}$, $\dot{\boldsymbol{\Psi}}_\mathbf{q}$, by doing this a `trajectory_info` is created. Based on the automatically and manually constructed elements the feedforward `controller.py` solves the equation (36) for control torques $\mathbf{u}$ as well as the positions, velocities and Lagrange multipliers $\mathbf{q}$, $\dot{\mathbf{q}}$, $\boldsymbol{\lambda}$.

It is vital to set configure the co-simulation environments in a way that integration step of the controller matches the communication interval between Simulink and Adams. On top of that, `Discrete Computational Order - Simulink Leads Adams` should be set to `no`, ensuring the simulation starts in the correct order.

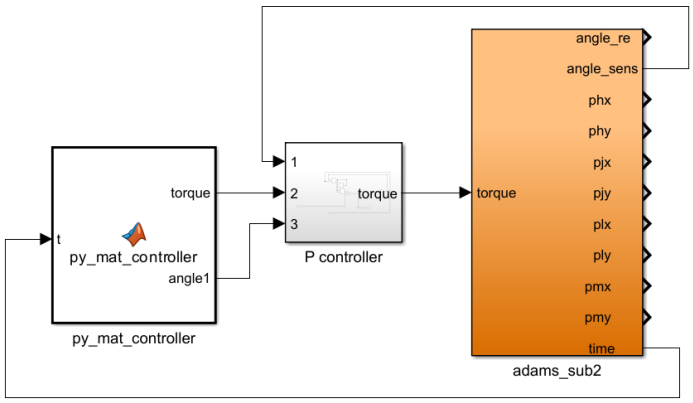


Figure 4.3. basic Simulink setup

Each simulation step dt python feedforward controller reads the previous step state information from a temporary file `tempstate.npy` and receives current simulation time from Simulink. It uses the information to solve the set of DAE-s. State variables calculated

in the previous step, $\mathbf{q_n}$, $\dot{\mathbf{q}}_\mathbf{n}$ are used explicitly, $\mathbf{q_n}$, $\dot{\mathbf{q}}_\mathbf{n}$, $\boldsymbol{\lambda_n}$, $\mathbf{u_n}$ are used as the initial computation guess. It overwrites new state variables in a temporary file and transfers them to a Simulnik model. There, after potential feedback correction of the torques, the final torque output is fed to Adams. It calculates the forward dynamics and responds back with the data from sensors set up during the configuration. The process is continued throughout the simulation. The python feedforward controller is a fully open loop one, it only receives the current time data, it is blind to the response of the system. The entire feedback loop is carried out in Simulnik.

Below are shown code snippets displaying the communication between Python code and Simulink environment. Of crucial importance is the explicit definition of data types used, in most cases `double` is sufficient. In this example `time_at_sim` is transferred to Python, and `data_out` is received by Simulink.

Listing 1. MATLAB function, communication with python script

```
function [torque, angle1]  = py_mat_controller(t)
    torque = 0;
    angle1 = 0;
    time = t;
    coder.extrinsic('pyrunfile');
    result = pyrunfile("controller.py", "data_out", time_at_sim = time);
    result = double(result);
    torque = result(17);
    angle1 = result(3);
end
```

Listing 2. Highly simplified python script fragment, communication with Simulink

```
def run_sim(t):
    #
    ...
    #
    q0_ = solution[:6].reshape(-1, 1)
    v0_ = solution[6: 12].reshape(-1, 1)
    lamb0_ = solution[12: 16].reshape(-1,1)
    u0_ = solution[16]
    save_data("tempstate.npy", solution)
    return np.hstack((q0_.flatten(), v0_.flatten(),
            lamb0_.flatten(), u0_.flatten())).tolist()

t = time_at_sim
data_out = run_sim(t)
```

5. Results

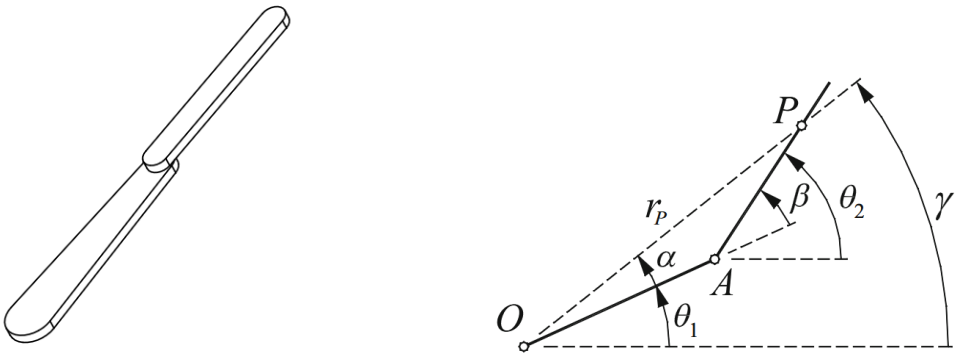
In this section, two mechanisms were investigated: an underactuated rotational manipulator arm and a four-link closed-loop mechanism. The goal of it was to observe and analyze different properties and phenomena of the mentioned mechanisms, as well as the control properties. For each system, a simple feedback loop containing a proportional term was added to observe its influence on the feedforward control system. Firstly, each mechanism was designed in Siemens NX CAD software, then exported to Hexagon Adams. As mentioned in the previous section, a Simulink model of the control system was exported. The feedforward control was performed by Python script. To solve the system of equations (36) a `scipy.optimise.root` function was used. In the first example, the solver method set to `'hybr'`(based on modified Powell method) yielded fast and reliable results. However, in the second example, it was either not able to converge at all or at some point in the simulation. Another method `'lm'`, based on Levenberg–Marquardt algorithm, was chosen.

5.1. Rotational manipulator arm

The mechanism is composed of two links connected by one frictionless rotational joint with a rotational spring-damper. The control input to the system was torque applied on the base of the system. The mechanism is underactuated, it has 2 DOF while having one input. Depending on the simulation, the gravitational force was taken into account or not. The mass properties were automatically assigned in Adams, assuming the density of steel. The mechanical properties of the system are shown below.

Table 5.1. Mechanical properties of the manipulator arm

Mass	$M_1 = 9.147 \times 10^{-2}\, kg$	$M_2 = 5.100 \times 10^{-2}\, kg$
Center moment of inertia	$I_1 = 1.028 \times 10^{-4}\,\mathrm{kg \cdot m^2}$	$I_2 = 4.349 \times 10^{-5}\,\mathrm{kg \cdot m^2}$
Length	$l_1 = 0.1\, m$	$l_2 = 0.09\, m$



(a) **(b)**

Figure 5.1. Manipulator arm physical and simmplified model

The control goal of this system was tracking of the angular position of the point P located on the underactuated arm. Proposed in [9] was the use of linearly approximated angle function. Considering the naming convention on 5.1 the sine rule was applied to the triangle $\triangle_{OAP}$.

$$\frac{\sin(\alpha)}{AP} = \frac{\sin(\beta)}{OP} \tag{38}$$

Assuming the angles α, β are small,

$$\frac{\alpha}{AP} = \frac{\beta}{OP}. \tag{39}$$

Then considering the fact that,

$$\gamma = \theta_1 + \alpha, \tag{40}$$

$$\beta = \theta_2 - \theta_1, \tag{41}$$

$$OP \approx OA + AP, \tag{42}$$

the angle output γ is described.

$$\gamma = (1-\kappa)\theta_1 + \kappa\theta_2 \tag{43}$$

Where $\kappa = \frac{AP}{OA+AP}$ is a constant coefficient. The servo constraint $\mathbf{\Psi}$, its Jacobian $\mathbf{\Psi_q}$, and its time derivative $\dot{\mathbf{\Psi}}_\mathbf{q}$ are shown below.

$$\mathbf{\Psi} = (1-\kappa)\theta_1 + \kappa\theta_2 - \tilde{y}(t) = \mathbf{0} \tag{44}$$

$$\mathbf{\Psi_q} = \begin{bmatrix} 0 & 0 & (1-\kappa) & 0 & 0 & \kappa \end{bmatrix} \tag{45}$$

$$\dot{\mathbf{\Psi}}_\mathbf{q} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{46}$$

The $\tilde{\mathbf{y}}(t)$ function was chosen to perform a point to point maneuver. It must have smooth acceleration profile in order to enable exact trajectory tracking, obviously due to inability of mechanical systems to perform instantaneous acceleration. In [9] proposed was a polynomial in a form:

$$\tilde{y}_s(t) = d \cdot \left[126\left(\frac{t}{\tau}\right)^5 - 420\left(\frac{t}{\tau}\right)^6 + 540\left(\frac{t}{\tau}\right)^7 - 315\left(\frac{t}{\tau}\right)^8 + 70\left(\frac{t}{\tau}\right)^9 \right] \tag{47}$$

where τ is the time between start and stop of the movement and d is a displacement. The desired trajectory was composed of the smooth movement period , followed by

a stationary final position maintenance t_S.

$$\tilde{y}(t) = \begin{cases} \tilde{y}_s(t), & \text{if } t \leq \tau \\ d, & \text{if } t > \tau \end{cases} \tag{48}$$

To construct the matrix $\boldsymbol{\xi}$, smooth position function derivatives were calculated numerically for convenience. The five point stencil method with a step $h = 1e-4$ was used to decrease the numerical error. The first and second derivatives are described as follows:

$$f'(x) \approx \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h}, \tag{49}$$

$$f''(x) \approx \frac{-f(x+2h) + 16f(x+h) - 30f(x) + 16f(x-h) - f(x-2h)}{12h^2}. \tag{50}$$

For all simulations performed, the control curve coefficients were: $d = 2\pi, \tau = 2$, $t_s = 1s$, meaning the point P was supposed to follow the angle trajectory and perform a full rotation in 2s, then maintain it for another 1s.

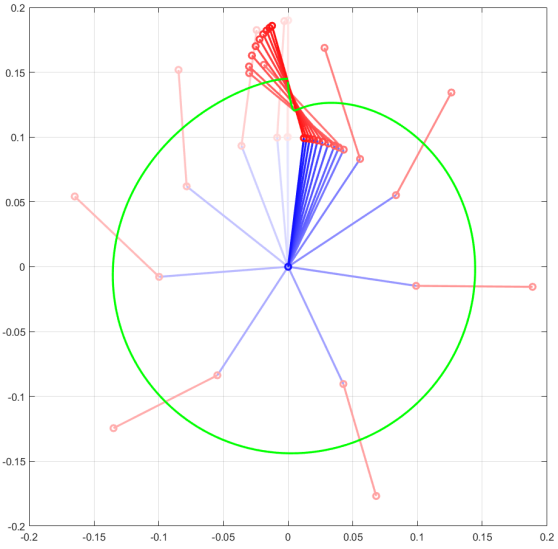
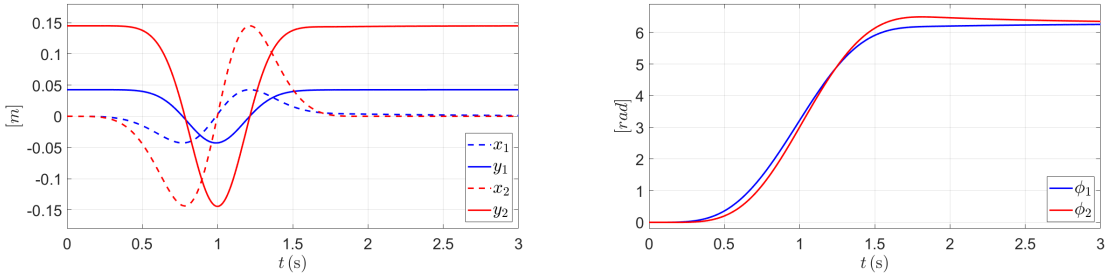


Figure 5.2. Position of the mechanism through time

5.1.1. Primary configuration

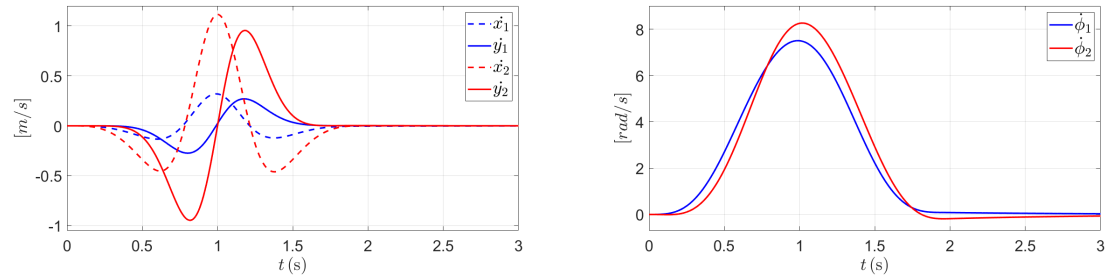
In this example, motion of the point P, located in the center of mass of the underactuated link was controlled. The goal was to perform a full rotation with a complete stop at the end of the movement. Spring stiffness was set to $k = 0.005\,\frac{Nm}{rad}$, $c = 0.005\,\frac{Nm \cdot s}{rad}$, the assumption of the small deformation angle was assured. The gravitational acceleration was set to $0\frac{m}{s^2}$, and integration step was $h = 0.001s$. The following graphs 5.3, 5.4 show the resulting movement positions and velocities of the center of mass of each body. Plots

5.5 show the torque input **u** and calculated by the controller software and Lagrange multipliers corresponding to forces acting on bodies due to mechanical constraints. It can be deducted, that the mechanism follows the desired trajectory closely however, the tracking precision is analyzed in the next simulation more in depth.



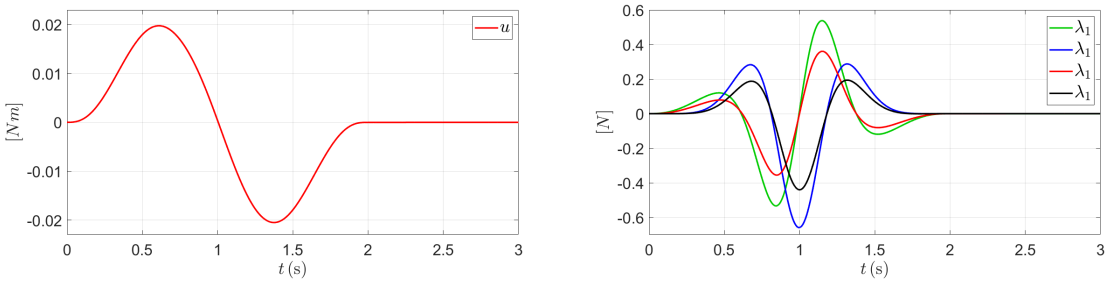
(a) **(b)**

Figure 5.3. Coordinates **q**



(a) **(b)**

Figure 5.4. Velocities $\dot{\mathbf{q}}$



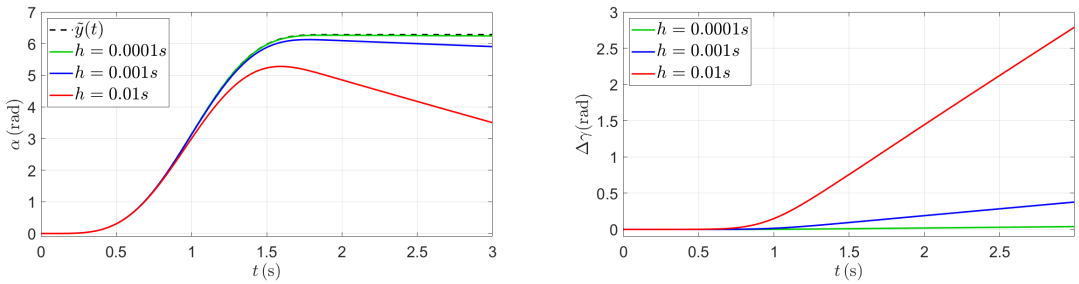
(a) **(b)**

Figure 5.5. Control input **u** and Lagrange multipliers $\boldsymbol{\lambda}$

5.1.2. Integration step size analysis

In this section a influence of integration step length on the trajectory tracking was investigated. All other mechanical properties of the model were left unchanged. The

backward Euler method is very sensitive to the step size, it can be seen on the plots below. The error seem to grow linearly with time, however, not from beginning. Considering that the mechanical system in the analyzed configuration is relatively stable, it seems to align with the assumption of RMS error being $\mathcal{O}(h)$.



(a) **(b)**

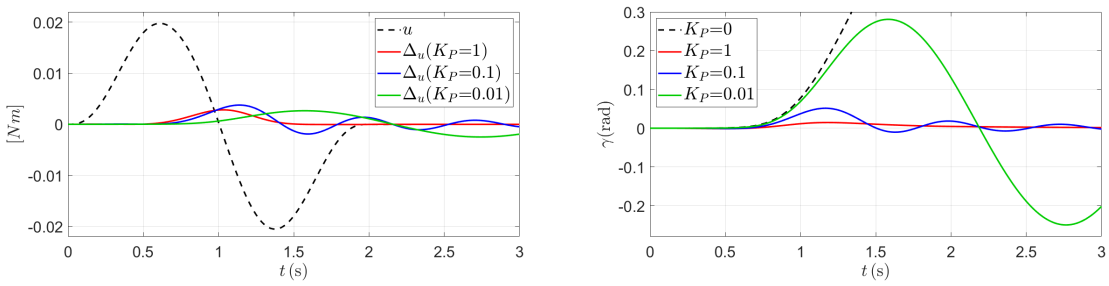
Figure 5.6. Position of the point P (a), tracking error (b)

5.1.3. Feedback loop

In order to decrease the error caused by numerical computation inaccuracy a so- called two-design degrees of freedom control schema with feedback, described previously, was assembled. The difference between calculated and measured angular position of the manipulator Δ_α was being measured. Only the proportional term P was used. The final control input u was computed.

$$u = u_c + \Delta_\alpha \cdot K_P \tag{51}$$

For the integration step $h = 0.005s$, three proportional constants were tested. The plot (a) in 5.7 shows the input portion added by the controller and the final torque output. Plot (b) shows the trajectory tracking error. It is apparent that the highest gain $P = 1$ achieves the best properties, minimizes the error while adding the least torque to the system.



(a) **(b)**

Figure 5.7. Influence of the P controller

5.1.4. Highly flexible model

In this simulation, a vastly different configuration was studied. The spring stiffness was decreased to k = 0.001 $\frac{Nm}{rad}$ and damping was set to zero, $c = 0\,\frac{Nm \cdot s}{rad}$. A very small integration step was set to $h = 0.0001s$ and proportional term left at $K_P = 1$.
Lets first appreciate the the qualities of model based control algorithms. Figure 5.8 shows the control torque applied and the minimal support of the feedback loop. As it can be seen on graph 5.9 the trajectory tracking is very precise $(\tilde{y}(t), \gamma)$.
However, the position control is based on the linearly approximated angle γ. Due to the decreased spring stiffness the mechanism deforms significantly beyond the linearly approximated angle and the actual angle between the base and point P is shown by $\tilde{\gamma}$. The relative angle β between links can be observed on the last graph. It can be concluded, that even though the trajectory tracking is very precise the mechanism moves in a potentially unintended way.

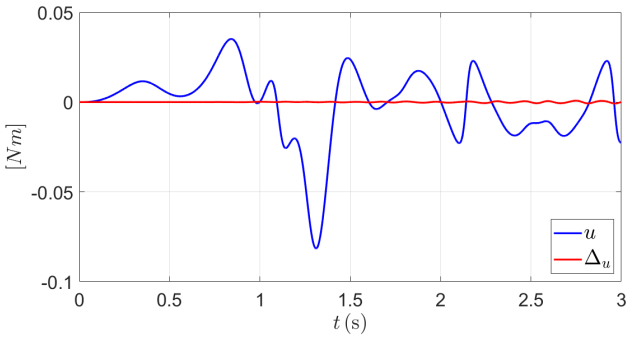
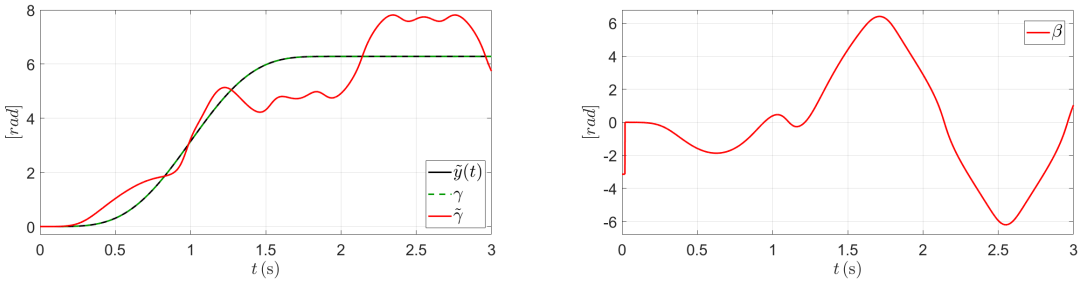


Figure 5.8. Applied torque



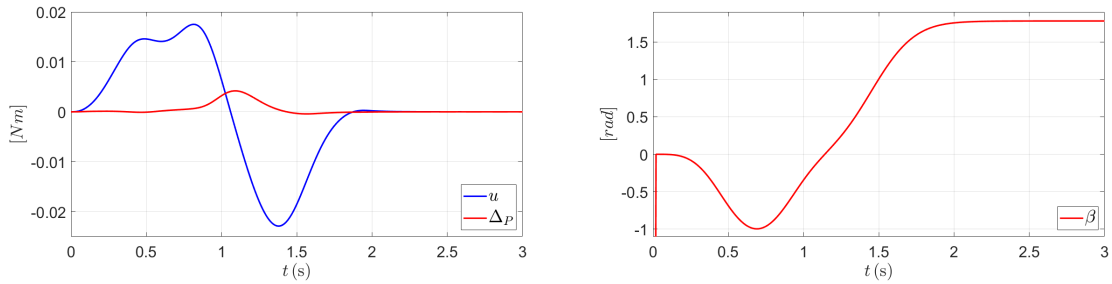
(a) **(b)**

Figure 5.9. Trajectories of the point P, relative angle

5.1.5. Spring-damper properties

This time the integration step was set to $h = 0.005s$ and proportional term stayed at $K_P = 0$. Firstly, a a spring stiffness was decreased to $k = 0\,\frac{Nm}{rad}$ and only damping was left set to $c = 0.001\ \frac{Nm \cdot s}{rad}$. It can be seen that the control torque is very smooth and due to

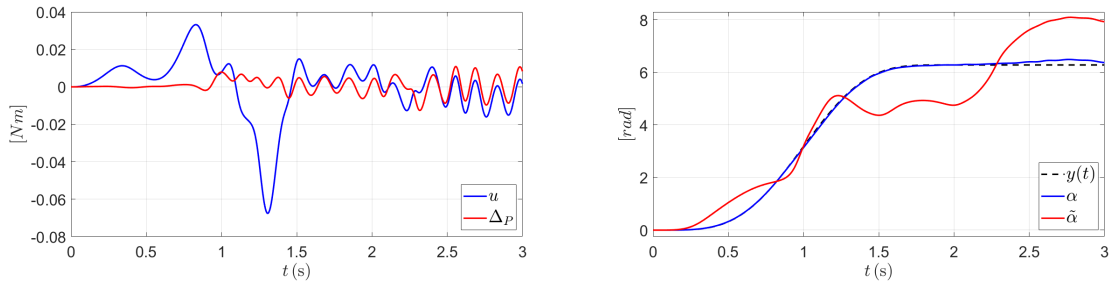
a lack of force coming from a spring, the underactuated link does not return to its start position.



(a) **(b)**

Figure 5.10. Control torque and relative angle with only dampening effect

The following example is opposite, the damping was decreased to zero and only the spring was left with the stiffness of $k = 0.001\ \frac{Nm}{rad}$. Trajectory tracking is still precise, however the P controller starts introduce unnecessary oscillation.

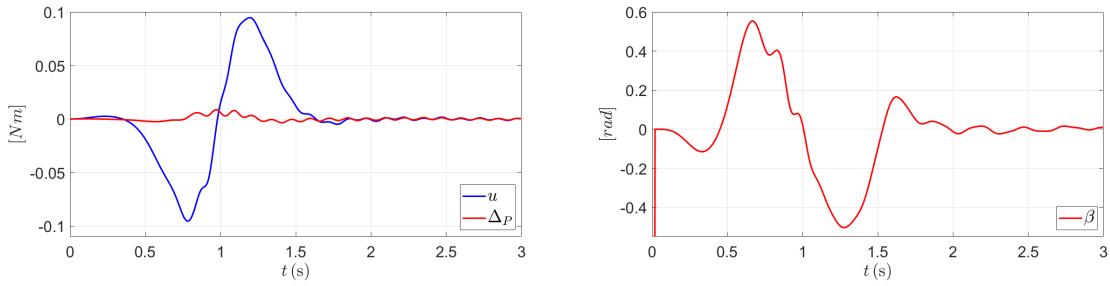


(a) **(b)**

Figure 5.11. Instability caused by low spring stiffness

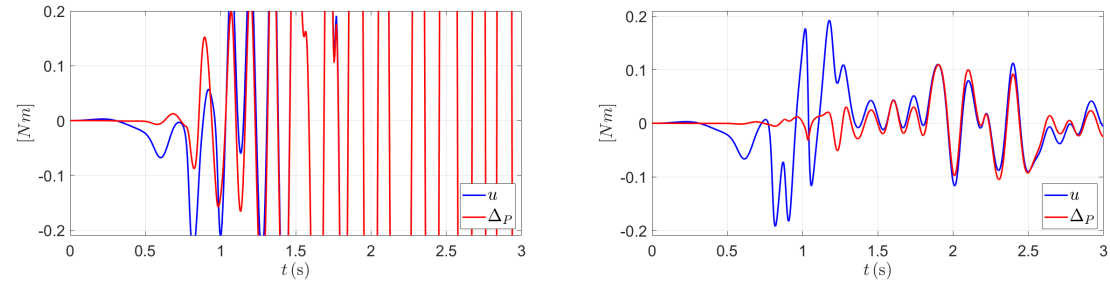
5.1.6. Gravitational force

The influence of the gravitational force was discussed in the following example. Firstly, a spring constant was set to $k = 0.03\ \frac{Nm}{rad}$while damping coefficient was left at $c = 0\ \frac{Nm \cdot s}{rad}$. The beginning of the movement show interesting phenomena. At the start of the motion, a delicate positive torque is applied. It pushes the mechanism left from equilibrium point. At the same time β drops temporarily below 0. Then, the torque input start acting like a brake to decrease the speed of the mechanism, meaning it acts in the opposite direction to all previously discussed examples. Due to the initial added torque and the complete lack of dampening in the system, the final oscillation continues after reaching the final position.
By decreasing the string constant slightly to $k = 0.02\ \frac{Nm}{rad}$ mechanism becomes unstable and uncontrollable.



(a) **(b)**

Figure 5.12. Stable control with gravity forces

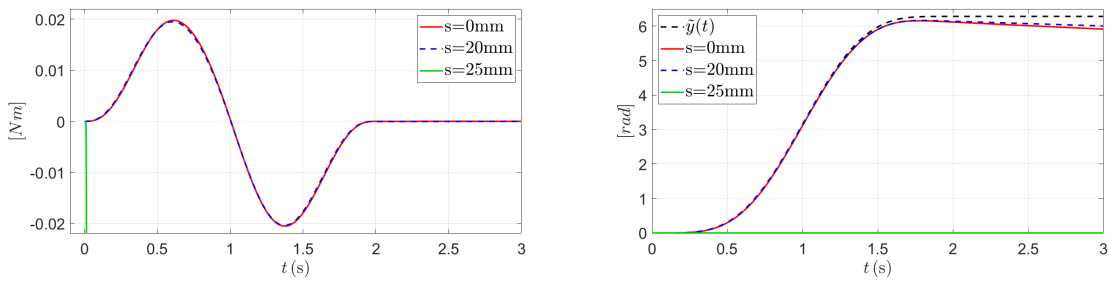


(a) **(b)**

Figure 5.13. Unstable control with gravity forces

5.1.7. Position of the control point P on the underactuated link

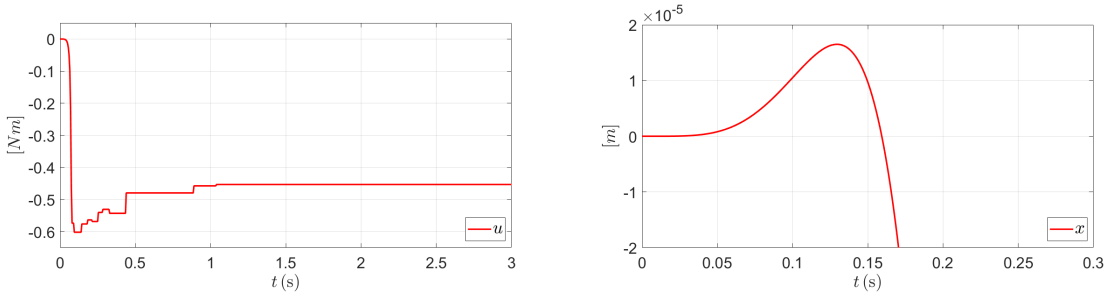
One important phenomena of the analyzed example is the influence of the position of the tracked point P on the stability of the algorithm So far, the point was always located in the center of mass of the outer link, being also its geometrical center. By increasing the distance of the tracking point from the center by 20mm the feedforward algorithm stayed stable, even increased the tracking precision slightly. However, increasing the position by further 5mm made the algorithm unstable and unable to converge.



(a) **(b)**

Figure 5.14. Trajectory tracking depending on the placement of the point P

To understand this behaviour it is worth looking at the initial applied torque sign. Lets remind that the arm is supposed to move in a counterclockwise direction, therefore it is logical to assume that torque with a positive sign should be applied however, the contrary is observed. Graph 5.15 (b) is a closeup of the x position of the endpoint of the outer arm, made during another successful simulation. It turns out that at the beginning of the movement the outer part of the link travels briefly to the right. Therefore it can be deduced, that due to this phenomena, if point P is set far enough, the controller tries to offset this initial move by applying a negative torque, which is obviously incorrect and leads to a subsequent calculation failure.



(a) **(b)**

Figure 5.15. Control torque and closeup of the horizontal position of the end point

This system property is easier to observe when mechanism is described in a minimum set of coordinates, in [9]. It turns out that the stability of the solution depends on many factors, position of the point P and the relative angle of the arms β, among others.

5.2. Fully actuated 2-DOF mechanism

In the following section, a mechanism composed of 4 links connected by rotational joints was discussed. Both ends of the structure were connected to a stationary motors being used as torque control inputs. The mechanism has 2 DOF, also 2 control inputs, hence it can be classified as fully actuated multibody system. Gravitational acceleration was neglected throughout the analysis.

Table 5.2. Mechanical properties of the 4-link mechanism

Mass	$M_i = 0.065\,kg$
Center moment of inertia	$I_i = 9 \times 10^{-5}\,\mathrm{kg \cdot m^2}$
Length	$l_i = 0.15\,m$

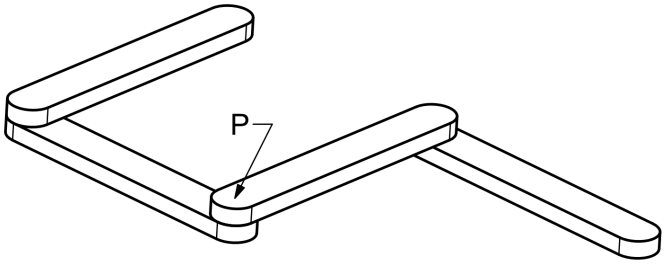


Figure 5.16. 2 DOF mechanism

The goal was to control the point P located in the middle of the center joint. Prescribed was a movement along a Lissajous curve. Since both x and y coordinates of the point P can be controlled independently a following trajectory was proposed

$$\tilde{\boldsymbol{\gamma}}(t) = \begin{bmatrix} A \cdot sin(at + \delta) \\ B \cdot sin(bt) \end{bmatrix}, \tag{52}$$

where constant parameters were set to: $A = 0.05m$, $B = 0.05m$, $\delta = 0$, $a = 3\pi/2$, $b = \pi$. The described curve does not have smooth acceleration, however. By multiplying the time variable by the smoothing curve described in the previous example a needed result was achieved. The final prescribed trajectory looks as follows.

$$\tilde{\boldsymbol{\gamma}}(t) = \begin{bmatrix} 0.005 \cdot sin(\frac{3\pi}{2} \cdot \tilde{y}_s(t) \cdot t) \\ 0.005 \cdot sin(\pi \cdot \tilde{y}_s(t) \cdot t) \end{bmatrix} \tag{53}$$

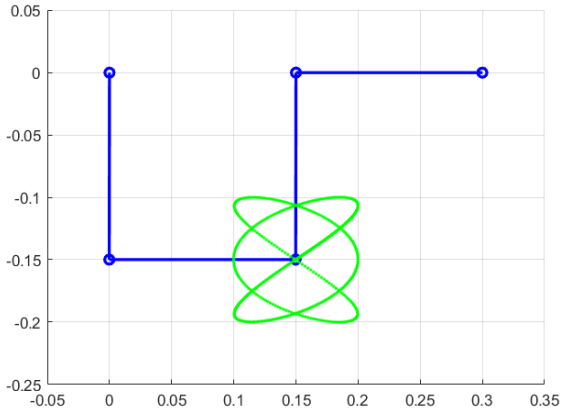


Figure 5.17. Lissajous curve

Both first and second derivatives were achieved by previously discussed numerical differentiation. Next, necessary elements $\boldsymbol{\Psi}, \boldsymbol{\Psi}_{\mathbf{q}}, \dot{\boldsymbol{\Psi}}_{\mathbf{q}}$ are defined in a similar way to me-

chanical constraints. It is assumed that point P lies on the third body.

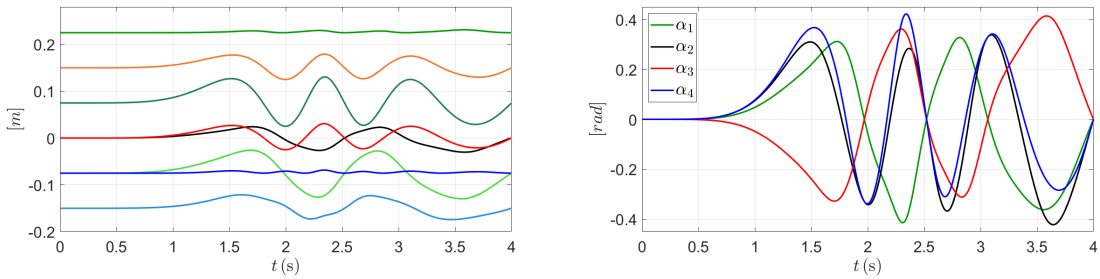
$$\mathbf{\Psi} = \tilde{\mathbf{q}}_3 + \mathbf{R}(\alpha_3)\mathbf{s}_P^{(3)} - (\tilde{\boldsymbol{\gamma}}(t) + \mathbf{P}_0) = \mathbf{0} \tag{54}$$

$$\mathbf{\Psi}_\mathbf{q} = \begin{bmatrix} \mathbf{0}_{2\times 6} & \mathbf{I}_{2\times 2} & \boldsymbol{\Omega}\boldsymbol{R}(\alpha_3)\mathbf{s}_P^{(3)} & \mathbf{0}_{2\times 3} \end{bmatrix} \tag{55}$$

$$\dot{\mathbf{\Psi}}_\mathbf{q} = \begin{bmatrix} \mathbf{0}_{2\times 6} & \mathbf{I}_{2\times 2} & \boldsymbol{\Omega}\boldsymbol{\Omega}\boldsymbol{R}(\alpha_3)\mathbf{s}_P^{(3)}\dot{\alpha}_3 & \mathbf{0}_{2\times 3} \end{bmatrix} \tag{56}$$

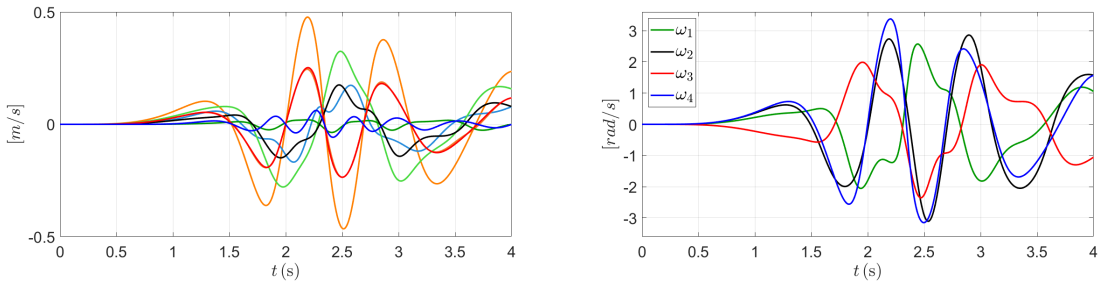
5.2.1. Basic configuration

Int the primary configuration the integration step size was set to $h = 0.005s$. The goal was to test the tracking properties and visualize all the calculated parameters. Firstly $\mathbf{q}$ positions and orientations are shown, followed by their derivatives $\dot{\mathbf{q}}$. At the end Lagrange multipliers and torque control inputs are presented. The final achieved trajectory was shown in picture 5.17.



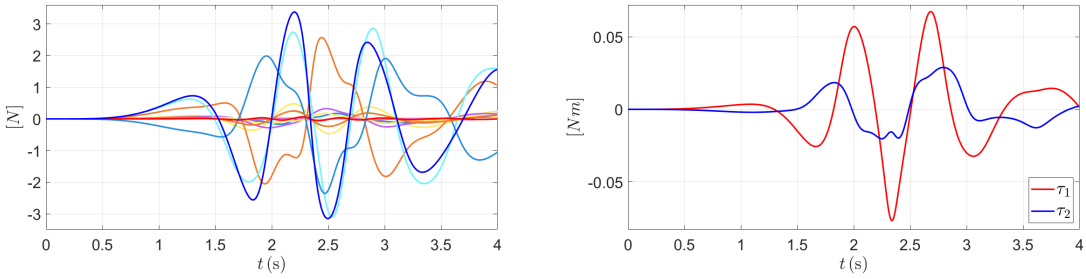
(a) **(b)**

Figure 5.18. 4- link mechanisms positions



(a) **(b)**

Figure 5.19. 4- link mechanism positions

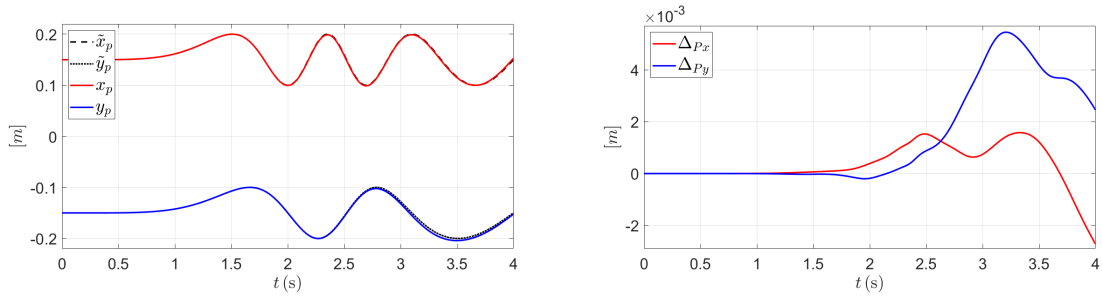


(a) **(b)**

Figure 5.20. 4- link mechanism lagrange multipliers and control torques

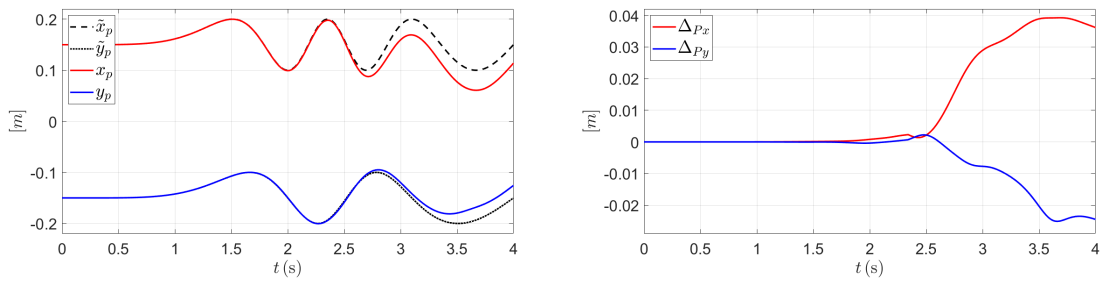
5.2.2. Integration step size analysis

In this section the influence of the integration step was checked. Three values were tested ($0.005s$, $0.01s$, $0.05s$). Unlike in the previous mechanism, global tracking error here does not rise linearly. It is due to the ability of the mechanism to move more freely because of the lack of springs and dampers. The interacting control torques and acceleration of the movement increase the error too. For small enough integration step the error increases in an predictable way. However, after a long enough movement time a chaotic behaviour is observed.



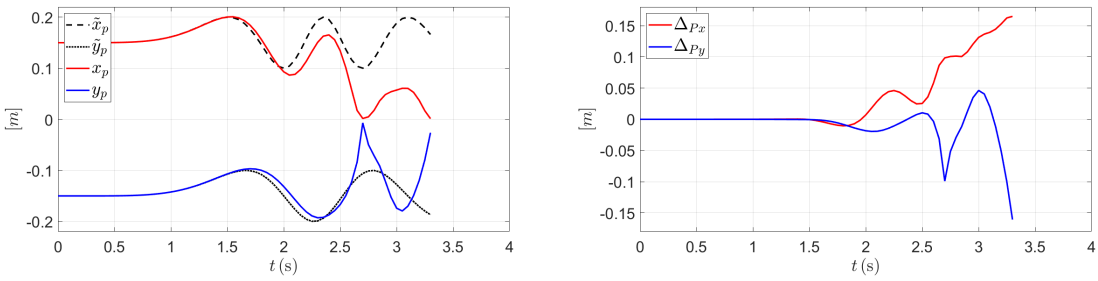
(a) **(b)**

Figure 5.21. Tracking position and accuracy difference, $h = 0.005s$



(a) **(b)**

Figure 5.22. Tracking position and accuracy difference, $h = 0.01s$



(a) **(b)**

Figure 5.23. Tracking position and accuracy difference, $h = 0.05s$

5.2.3. Electric motor

Instead of using torque values directly as control inputs, a solution using electric DC motor was proposed. It was made to observe the system properties under more realistic conditions and to test the controller accuracy. To achieve this, after calculating the control torque value, a corresponding voltage applied to motors needed to be calculated. A basic model of a DC electric motor was was proposed. Firstly a Kirchhoff's voltage law is applied, taking into consideration the motor induced backwards EMF V_e.

$$L\frac{dI}{dt} + RI = V - V_e \tag{57}$$

Where I is the current flowing through the motor, R is its resistance. The first term in the equation is relatively small, therefore it can be neglected. Next, the equation of the torque at the shaft of the motor is presented:

$$\tau_m = K_T I - B_m \omega_m \tag{58}$$

where, K_t is motor current-torque constant, B_m is a net friction coefficient and ω_m is the angular velocity of the motor shaft. Combining both equations a governing equation of motion of electric motor is introduced.

$$\tau_m = \frac{K_t}{R}(V - K_m \omega_m) - B_m \omega_m, \tag{59}$$

Table 5.3. DC motor parameters

Back EMF constant	$K_m = 7.68 \cdot 10^{-7} kg\, m^2$
Motor current-torque constant	$K_t = 5.3 \cdot 10^{-3} \frac{Nm}{A}$
Motor resistance	$R_m = 2.6\Omega$
Net friction coefficient	$B_m = 3.42 \cdot 10^{-6} \frac{Nms}{rad}$
Transmission gear ratio	$k_g = 70$

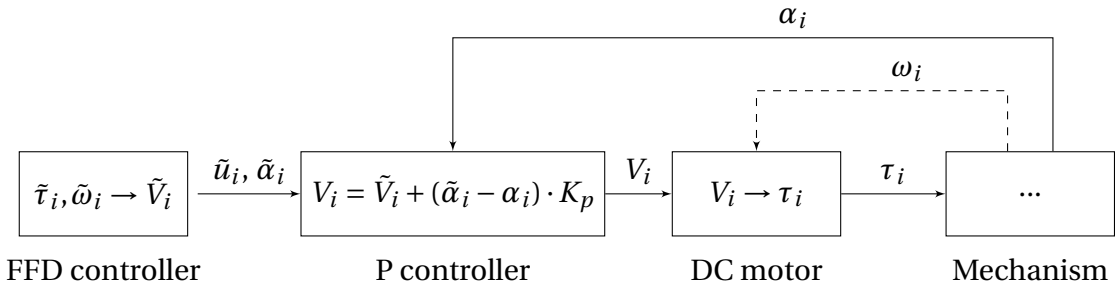
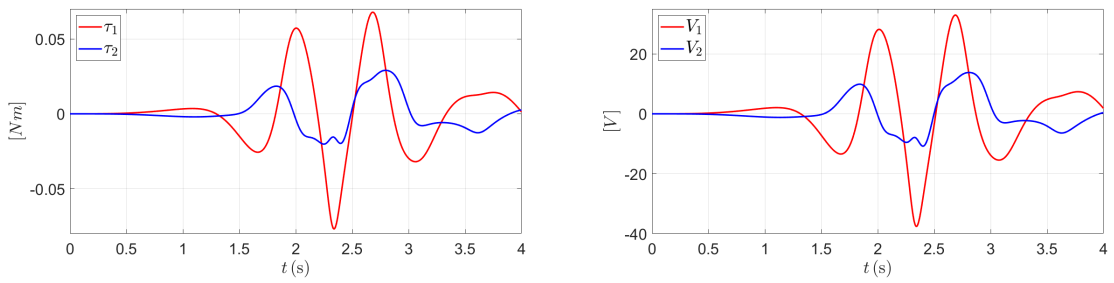


Figure 5.24. DC motor control structure

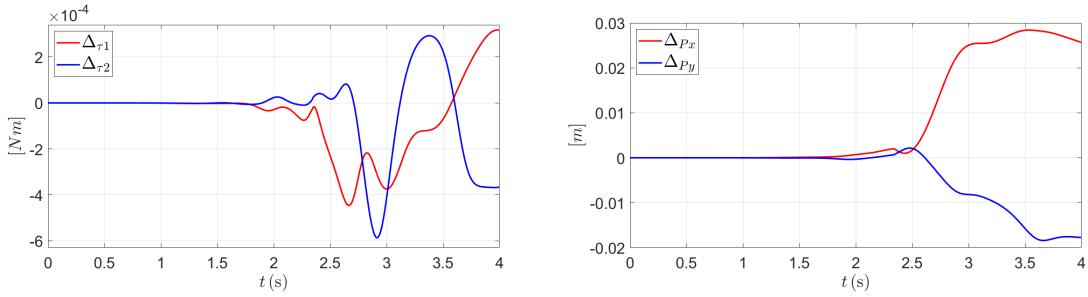
In the feedforward controller, the torque is calculated just as previously. Then, using the equation (59) a voltage applied to the DC motors $\tilde{V}_i$ are determined based on the calculated rotational speeds ω_i and motor constants. Transmission gear ration is taken into consideration and treated as an additional constraint. Next, a simple P controller adjusts the voltage based on the difference of the calculated and real angular positions of the outer links. In this example, the gain K_p was set to zero to observe just the influence of the electric motor. Further, a motor transforms the input voltages V_i into final control torques τ_i. It is considered that the DC motor model in FFD controller is precise. Of course in reality the angular speed of the motor is its inherit property, however, to visualize the simulation configuration ω_i is treated as a feedback from a mechanical system.

In the test example the same control task was required. Integration step size was set to $h = 0.01s$. Figures 5.25 shows the final applied torque and corresponding voltage applied to the motor. The shapes of the curves are similar, as expected. For the chosen configuration the torque calculated directly by a FFD controller and one applied by the motor does not vary greatly 5.26 (a). Interestingly the desired position is tracked with more precision than using the torques $\tilde{\tau}_i$ directly.



(a) **(b)**

Figure 5.25. Applied voltage V_i and corresponding torque τ_i



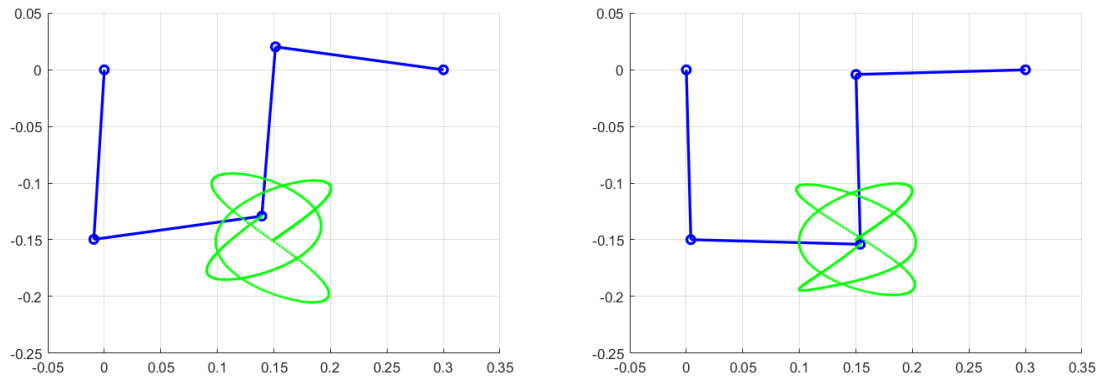
(a) **(b)**

Figure 5.26. Difference between motor and explicit torques, corresponding trajectory tracking performance

5.2.4. Model inaccuracy

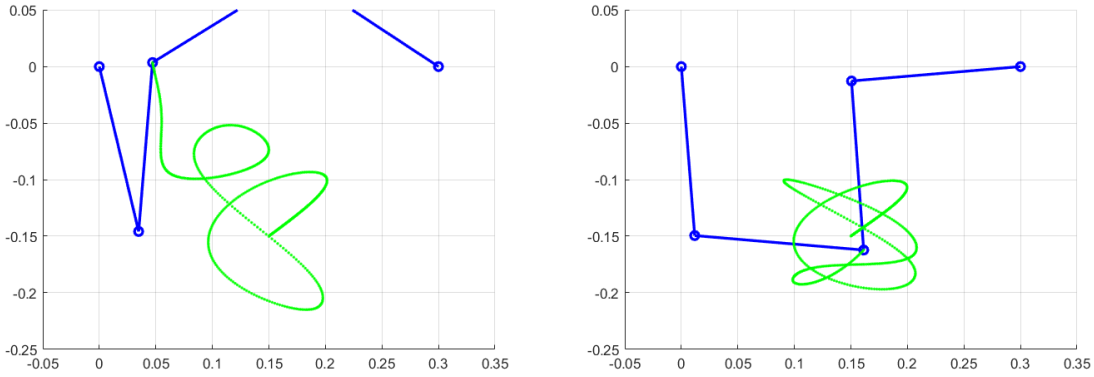
In the last example, an influence of the model inaccuracy was investigated. So far, the feedforward controller has always had exact information about the mechanical system. However, control solution is model-based and the feedback control is minimal, therefore any discrepancies may lead to substantial control errors. In real-world conditions rarely is it possible to account for all disturbances and collect the precise information about the mechanism.

In the following examples, the mass and moment of inertia of the second link was decreased by 10% and 30% accordingly. The information about that change was left unnoticed in the FFD model. Each case was tested with the feedback proportional controller with constants $K_p = 0$ and $K_p = 30$. Integration constant was left at $h = 0.01s$. A more qualitative approach was used this time to display the results.



(a) **(b)**

Figure 5.27. Mass increased decreased by 10%



(a) **(b)**

Figure 5.28. Mass increased decreased by 30%

5.3. Results discussion

Feedforward control using servo constraints has been analyzed. It yielded good quality tracking performance for both mechanisms, only if the integration time step h was sufficiently small. A simple feedback loop based on a proportional controller was able to consistently increase the tracking precision in many cases. Even if faced with slight model inaccuracies it proved to be effective. The definition of trajectory curve was discussed too, it was shown that careful thought should be drawn to systems with linearly approximated outputs as in certain conditions they can deviate significantly from real outputs. The durability of the control algorithm was also checked, it was found that trajectory tracking may not always be possible and is very mechanism- dependent. In general, it can be observed that each multibody system has its own unique phenomena that should be taken into consideration when designing the desired trajectory path.

6. Summary, conclusions, and future plans

The objective of investigating feedforward control in underactuated multibody systems using servo constraints was successfully achieved. The results from the test cases aligned well with expectations based on the literature review. The correctness of them was guaranteed additionally by the use of external multibody dynamics software- Adams. The developed toolkit robustness and faultlessness was tested on two separate mechanisms, fully actuated and underactuated one. However, there are many areas that would benefit from significant improvement. Firstly the proposed code efficiency is relatively low, numerous variable initialization during the simulation process could be omitted to ensure faster calculation. The toolkit could be extended to allow a wider spectrum of mechanical constraints such as translational joints or contact forces. What is more, an improved feedback controller could be implemented, either by using PID controller or by returning the state variables directly to the feedforward stage. Finally, Euler backward differentiation scheme could be improved wit the use of Runge- Kutta or multi-step methods.

The co-simulation structure functioned flawlessly with both mechanisms, demonstrating their robustness. It significantly reduced the time required to set up all necessary elements, further highlighting that the bodies were designed using CAD software, thereby facilitating real-world applications. Its flexibility and convenience were further demonstrated by the seamless integration of the DC motor model, as well as the ability to add simple animations of the mechanism's movement. Additionally, the use of Adams' built-in GUI provided further advantages. Full control over all simulation parameters, such as integration step and mass properties, was also possible and successfully tested. Finally, the validity of the results was confirmed not only by comparison with the literature [8], [9] but also by considering the results computed by Adams, which can be regarded as essentially faultless.

Bibliography

[1] B. He, S. Wang, and Y. Liu, "Underactuated robotics: A review", *International Journal of Advanced Robotic Systems*, vol. 16, no. 4, p. 1 729 881 419 862 164, 2019. DOI: 10.1177/1729881419862164.

[2] R. Tedrake, *Underactuated Robotics, Algorithms for Walking, Running, Swimming, Flying, and Manipulation*. 2023. [Online]. Available: https://underactuated.csail.mit.edu.

[3] P. Nikravesh, *Computer-aided Analysis of Mechanical Systems*. Prentice Hall, 1988, ISBN: 9780131642386. [Online]. Available: https://books.google.pl/books?id=nCagAAAACAAJ.

[4] P. E. Nikravesh, *Planar Multibody Dynamics: Formulation, Programming with MATLAB®, and Applications, Second Edition*, 2nd ed. CRC Press, 2018. DOI: 10.1201/b22302. [Online]. Available: https://doi.org/10.1201/b22302.

[5] R. Seifried, *Dynamics of Underactuated Multibody Systems: Modeling, Control and Optimal Design* (Solid Mechanics and Its Applications). Springer International Publishing, 2013, ISBN: 9783319012285. [Online]. Available: https://books.google.pl/books?id=91a6BAAAQBAJ.

[6] S. Drcker, "Servo-constraints for inversion of underactuated multibody systems", Ph.D. dissertation, 2022.

[7] W. Blajer and K. Kolodziejczyk, "Control of underactuated mechanical systems with servo-constraints", *Nonlinear Dynamics*, vol. 50, pp. 781–791, Oct. 2007. DOI: 10.1007/s11071-007-9231-4.

[8] R. Seifried and W. Blajer, "Analysis of servo-constraint problems for underactuated multibody systems", *Mechanical Sciences*, vol. 4, pp. 113–129, Feb. 2013. DOI: 10.5194/ms-4-113-2013.

[9] W. Blajer, R. Seifried, and K. Kolodziejczyk, "Servo-constraint realization for underactuated mechanical systems", *Archive of Applied Mechanics*, vol. 85, Feb. 2015. DOI: 10.1007/s00419-014-0959-2.

[10] W. Blajer and K. Kołodziejczyk, "A geometric approach to solving problems of control constraints: Theory and a dae framework", *Multibody System Dynamics*, May 1, 2004.

[11] W. Blajer and K. Kolodziejczyk, "Improved dae formulation for inverse dynamics simulation of cranes", vol. 25, pp. 131–143, Feb. 2011. DOI: 10.1007/s11044-010-9227-6.

[12] K. Atkinson, *An Introduction to Numerical Analysis*, 2nd. Wiley, 1991, p. 720, ISBN: 978-0-471-62489-9.

[13] S. L. Campbell, "High-index differential algebraic equations", *Mechanics of Structures and Machines*, vol. 23, no. 2, pp. 199–222, 1995. DOI: 10.1080/08905459508905235.

[14] [Online]. Available: https://docs.scipy.org/doc/scipy/index.html.

[15] I. Fantoni and R. Lozano, *Non-linear Control for Underactuated Mechanical Systems* (Communications and Control Engineering). Springer, 2002. DOI: 10.1007/978-1-4471-0177-2. [Online]. Available: https://doi.org/10.1007/978-1-4471-0177-2.

[16] S. Marschner, *Cs3220 lecture notes: Singular value decomposition and applications*, Cornell University, 5–7 April 2010, 2010.

[17] [Online]. Available: https://math.libretexts.org/Bookshelves/Differential_Equations/Numerically_Solving_Ordinary_Differential_Equations_(Brorson)/01%3A_Chapters/1.03%3A_Backward_Euler_method.

[18] [Online]. Available: https://www.pydy.org/.

[19] [Online]. Available: https://pybullet.org/wordpress/.

[20] [Online]. Available: https://drake.mit.edu/.

List of Symbols and Abbreviations

List of Figures

List of Tables

List of Appendices

Appendix 1. Co- simulation setup

CAD Software:

- Create single-body models.
- Compose models into an assembly.
- Save the assembly as a Parasolid file.
- For Siemens NX, use the following steps:

```
File -> Export -> Parasolid -> Select objects in the assembly...
```

Adams:

- Change units to: [m, kg, N, s, Rad, Herz]
- Import the Parasolid file:

```
File -> Import -> Parasolid
File to read: <filename>
Model name: <right-click -> Browse...>
OK
```

- Add Joints and Forces.
- Create variables for tracking and controlling:

Example: Tracking Marker Variables Rename a marker to `Control_Point` and define state variables:

```
Elements -> System elements -> Create state Variable:
    Name: "X_Point"; F: "DX(Control_Point)"
    Name: "Y_Point"; F: "DY(Control_Point)"
    Name: "VX_Point"; F: "VX(Control_Point)"
    Name: "VY_Point"; F: "VY(Control_Point)"
```

Example: Torque Control Define torque control variables:

```
Elements -> System elements -> Create state Variable:
    Name: "Torque_1"; F: "0"
Modify First Force: Function: "VARVAL(.MODEL_1.Torque_1)"

Elements -> System elements -> Create state Variable:
    Name: "Torque_2"; F: "0"
Modify First Force: Function: "VARVAL(.MODEL_1.Torque_2)"
```

Exporting the Model for Matlab/Simulink

```
Plugins -> Controls -> Plant Export
    Name: ".MODEL_1.Controls_Plant_1"
    Input Signals: Torque_1, Torque_2
```

```
Output Signals: TIME, X_Point, Y_Point, VX_Point, VY_Point
Target Software -> Matlab
OK
```

Exported Files: Example directory: `C:\PI\4bar\adams_files`

- `aviewAS.cmd`
- `Controls_Plant_1.adm`
- `Controls_Plant_1.cmd`
- `Controls_Plant_1.m`
- `Controls_Plant_1.x_t`

The `Controls_Plant_1.adm` file contains the model data. Copy it to a Python project and save it as `model_data.txt`.

Matlab/Simulink

- Open `Controls_Plant_1.m` in Matlab.
- Change the working directory to the one containing Adams files.
- At the Matlab prompt, run:

```
>> Controls_Plant_1
```

- The output will include actuator and sensor information, for example:

```
%%% INFO : ADAMS plant actuators names :
1 Torque_1
2 Torque_2
%%% INFO : ADAMS plant sensors   names :
1 TIME
2 X_Point
3 Y_Point
4 VX_Point
5 VY_Point
```

- Build the Simulink model:

```
>> adams_sys
```

Simulink Adjustments: In the outermost part of the Simulink model:

- Comment out the two red blocks (`S-Function`, `State-Space`).
- Add constant values for testing.

Custom Function for Control Use the following Matlab function to connect Simulink to Python code:

```
function y = fcn(u)
    pos_act = u(1);  % Position input from Adams
    pos_und = u(2);  % Position input from Adams
    time = u(3);  % Time input from Adams
    y = 0.0;
    coder.extrinsic('pyrunfile'); % needed setup function

    result = pyrunfile("for_control_blocks.py", "control_force", ...
                       pos_act=pos_act, pos_und=pos_und, time=time);

    y = double(result);
end
```

Remember all source files should be inside the Matlab folder.

In MSC Software block set the following options (the rest should stay unchanged):

- Animation mode: "batch". It is the fastest option of calculation, it sues data from .adm file. If set to "interactive" Adams View will open and visualise the mechanism, dat is taken from the Adams model itself.
- Discrede Computational order- Simulink leads Adams: "no".
- `ADAMS_communication_interval` should be the same as in Python code.

Appendix 2. Python DAE equation solving code snippet

```
import numpy as np
from scipy.optimize import root

def equations(vars):
    q_ = np.array(vars[:len(q0_)]).reshape(-1, 1)
    v_ = np.array(vars[len(q0_): len(q0_)*2]).reshape(-1, 1)
    lamb_ = np.array(vars[len(q0_)*2: len(q0_)*2 +
    len(lamb0_)]).reshape(-1, 1)
    u_ = np.array(vars[len(q0_)*2 + len(lamb0_): len(q0_)*2 +
    len(lamb0_) + len(u0_)]).reshape(-1, 1)

    eq1 = (q_ - q0_) - v_ * dt
    eq2 = a1(q_) @ (v_ - v0_) + a2(q_, v_, u_) * dt
    eq3 = b1(q_, v_, t) + b2(q_, v_, u_, lamb_)
    eq4 = c1(q_, v_) + c2(q_, v_, u_, lamb_)
    eq5 = psi(q_, t, tau_, disp_)
    eq6 = phi(q_)
    return np.concatenate([eq1.flatten(), eq2.flatten(),
    eq3.flatten(), eq4.flatten(), eq5.flatten(), eq6.flatten()])

initial_guess = np.hstack([
    q0_.flatten(), v0_.flatten(), lamb0_.flatten(), u0_.flatten()
])

solution_res = root(
    equations,
    initial_guess,
    method='lm',  # or 'hybr', 'lm', 'krylov'
)
```